



نام دانشجو :

رضا ندیمی

شماره دانشجویی:

832204022

دانشکده سامانه های هوشمند - مکترونیک

تمرینات مکترونیک 2 سری پنجم

تاریخ:

1405/03/14

استاد مربوطه:

دکتر تارویردی زاده

فهرست

فضای توسعه ی مورد استفاده با توجه به عدم دسترسی به مخازن نرم افزار بین المللی 2

1. یک پتانسیومتر روی پایه ADC1_IN1 قرار دهید و با چرخاندن آن، ولتاژ مربوطه را بصورت دیجیتال روی LCD نمایش دهید (روی ۵۰ عدد داده، میانگین گیری کرده و به سه روش Polling، وقفه و DMA؛ این تمرین را انجام دهید).

3

1.1. روش Poling 3

1.2. روش interrupt 8

1.3. روش DMA 14

2. تمرین فوق را برای پایه های ADC1_IN1 و ADC1_IN2 و ADC1_IN3، تکرار کنید. در این تمرین، بجای استفاده از چند پتانسیومتر، می توانید فقط یک پتانسیومتر در بُرد داشته باشید و خروجی آن را، هر بار به یکی از پایه های فوق، وصل نمایید. 21

2.1. روش Poling 21

2.2. روش interrupt 27

2.3. روش DMA 35

3. یک پتانسیومتر روی پایه ADC1_IN1 قرار دهید و با چرخاندن آن، ولتاژ مربوطه را بصورت دیجیتال به نرمافزار Matlab بفرستید (از طریق UART) و اعداد مربوطه را در این نرمافزار نمایش دهید. 42

4. تمرین فوق را برای پایه های ADC1_IN1 و ADC1_IN2 و ADC1_IN3، تکرار کنید. خروجی نرمافزار Matlab، علاوه بر نمایش عدد دیجیتال متناظر با پایه ADC، بایستی مشخص کند که این عدد، مربوط به کدام کانال از ADC است. 49

4.1. روش Poling 49

4.2. روش DMA 56

5. در یک مدار، توسط یک منبع تغذیه، ولتاژ در بازه $V[-3.3, 3.3]$ ، ایجاد نمایید. به نوعی با میکروکنترلر، این ولتاژها را در یکی از پایه های ADC خوانده و عدد مربوطه را روی LCD نمایش دهید. متناظر با عدد $V3.3-$ ، عدد 4095، روی LCD نمایش داده شود و متناظر با $V3.3$ ، عدد 4095، روی LCD نمایش داده شود. با تغییر ولوم منبع تغذیه در بازه $V[-3.3, 3.3]$ ، اعداد مربوطه بصورت خطی در بازه $[4095, -4095]$ ، تغییر نماید. دقت داشته باشید که میکروکنترلر مدنظر ما، دارای ولتاژ کاری. تا ۳٫۳ ولت است و برای حل این تمرین، لازم است ملاحظات مناسب طراحی مدار را در نظر بگیرید. 63

فضای توسعه ی مورد استفاده با توجه به عدم دسترسی به مخازن نرم افزار بین المللی

1. برای STM32 CubeMX پک F1 نسخه ی V1.8.0

2. برای Keil پک Keil.STM32F1xx_DFP.2.4.0

3. برای Keil پک ARM.CMSIS.4.5.0 متعلق به سال 2015 (آخرین نسخه ای که در نت ملی توانستم

پیدا کنم. الان سال 2026 می باشد یعنی چیزی حدود 11 سال پیش)

4. برای ارتباط سریال با میکرو کنترلر از virtual serial driver pro استفاده شده است که از لینک زیر
دانلود شده است.

<https://p30download.ir/fa/entry/65323/virtual-serial-port-driver>

1. یک پتانسیومتر روی پایه ADC1_IN1 قرار دهید و با چرخاندن آن، ولتاژ مربوطه را بصورت دیجیتال روی LCD نمایش دهید (روی ۵۰ عدد داده، میانگین گیری کرده و به سه روش Polling، وقفه و DMA؛ این تمرین را انجام دهید).

1.1. روش Poling

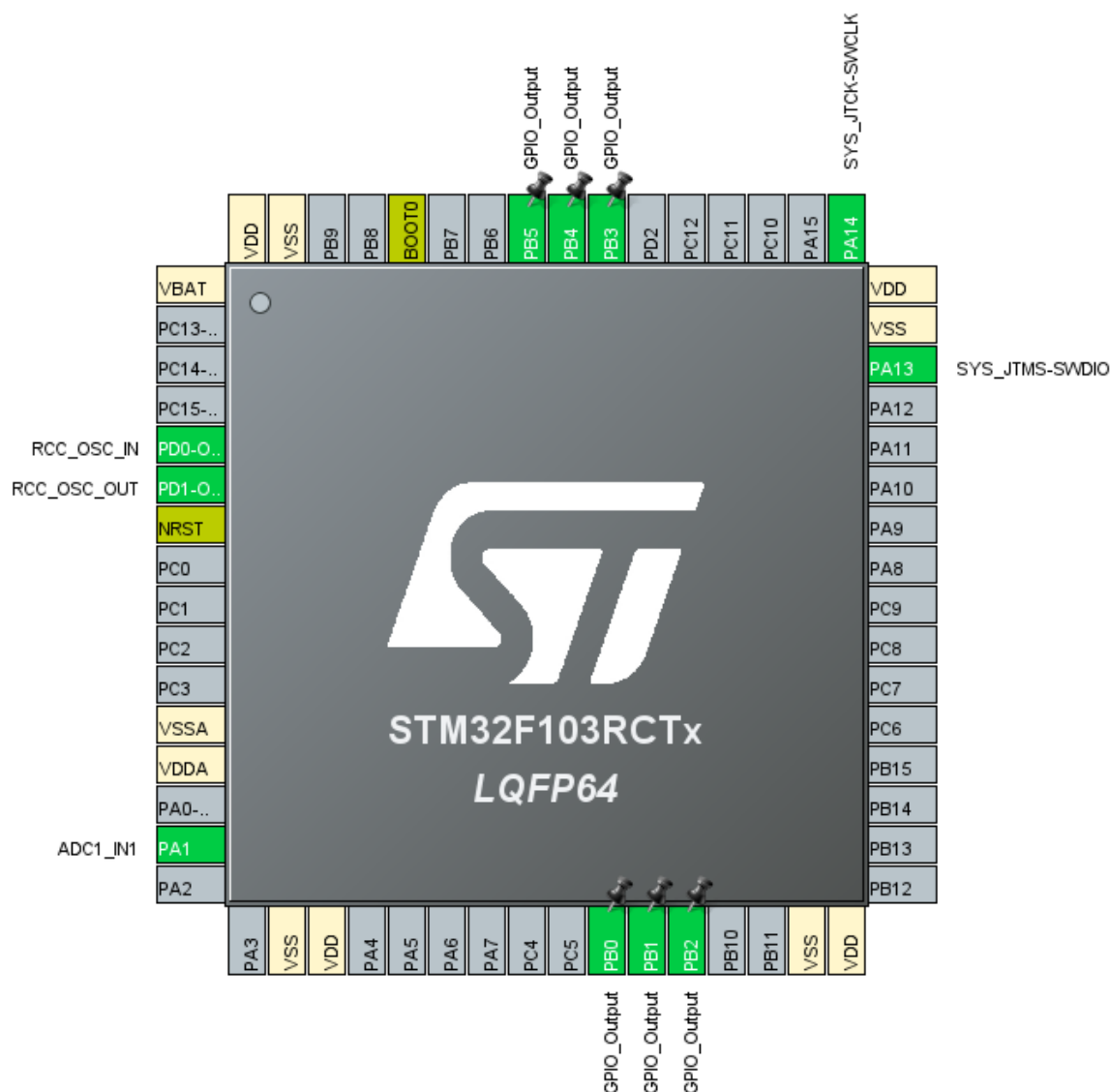
در این روش CPU به طور مداوم وضعیت را بررسی می کند.

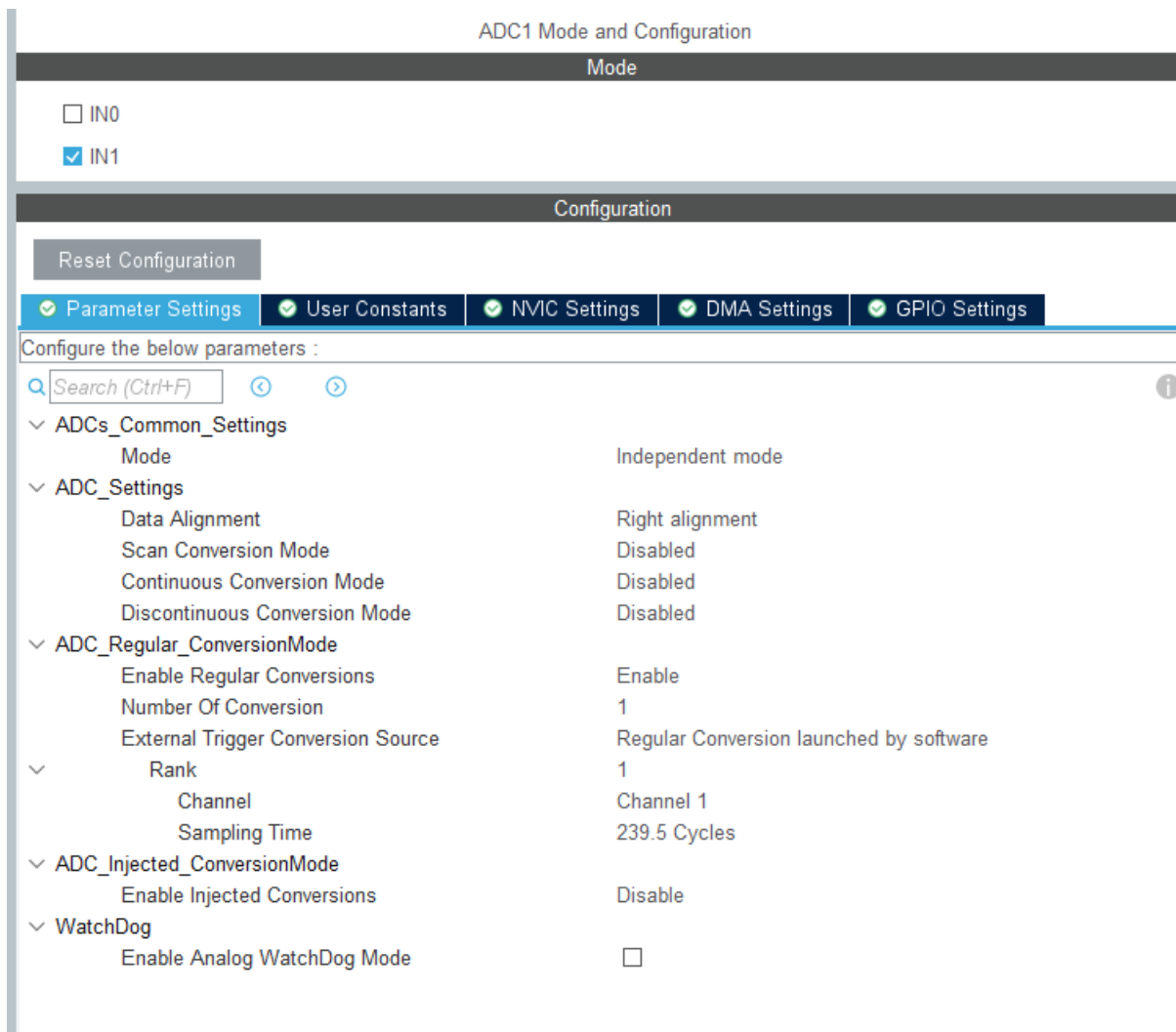
(بخش STM32CubeMX

میکرو کنترلر STM32F103RCT رو در نرم افزار انتخاب میکنیم.

پروگرامر رو به حالت Serial Wire و کلاک رو تنظیم میکنیم

STM32 رو مطابق تصاویر زیر پیکر بندی میکنیم.





بخش Keil (ii)

در این بخش فایل هدر lcd.h به پروژه اضافه شده است. این فایل برای استفاده از توابع کتابخانه‌ی LCD مورد نیاز است.

```
/* USER CODE BEGIN Includes */
#include "lcd.h"
/* USER CODE END Includes */
```

چند متغیر سراسری برای پروژه تعریف شده‌اند: یک بافر ۱۶ کاراکتری برای LCD به نام lcd_buffer، متغیر raw_adc برای نگهداری مقدار خام ADC، متغیر sum_adc برای جمع مقادیر نمونه‌ها، متغیر avg_adc برای میانگین نمونه‌ها و متغیر final_voltage برای ولتاژ نهایی. این متغیرها قرار است در بخش‌های دیگر برنامه استفاده شوند.

```

/* USER CODE BEGIN PV */
char lcd_buffer[16];
uint16_t raw_adc;
uint32_t sum_adc;
uint16_t avg_adc;
float final_voltage;
/* USER CODE END PV */

```

در این بخش مقداردهی اولیه‌ی LCD انجام می‌شود. آرایه‌های ports و pins برای مشخص کردن پین‌های داده و کنترل LCD تعریف شده‌اند. سپس با فراخوانی تابع Lcd_create یک هندل از نوع Lcd_HandleTypeDef ساخته می‌شود که در ادامه برای نمایش اطلاعات روی LCD به‌کار می‌رود.

```

/* USER CODE BEGIN 2 */
Lcd_PortType ports[] = { GPIOB, GPIOB, GPIOB, GPIOB };
Lcd_PinType pins[] = {GPIO_PIN_0, GPIO_PIN_1, GPIO_PIN_2, GPIO_PIN_3};
Lcd_HandleTypeDef lcd;
lcd = Lcd_create(ports, pins, GPIOB, GPIO_PIN_4, GPIOB, GPIO_PIN_5,
LCD_4_BIT_MODE);

/* USER CODE END 2 */

```

حلقه‌ی اصلی برنامه در این بخش پیاده‌سازی شده است. ابتدا ۵۰ نمونه از ADC با موفقیت خوانده می‌شود (برای اطمینان از صحت، فقط تبدیل‌های موفق شمارش می‌شوند). مجموع این ۵۰ نمونه محاسبه شده و سپس میانگین آن به‌دست می‌آید. برای محاسبه‌ی ولتاژ، از روش اعداد صحیح استفاده شده است: مقدار میانگین در ۳۳۰ ضرب و بر ۴۰۹۵ تقسیم می‌شود تا ولتاژ با دو رقم اعشار (به‌صورت یک عدد صحیح ضربدر ۱۰۰) به‌دست آید. سپس بخش صحیح و اعشاری ولتاژ جدا شده و با استفاده از sprintf و Lcd_string روی دو سطر LCD نمایش داده می‌شود. در پایان ۵۰۰ میلی‌ثانیه مکث ایجاد می‌شود تا نمایشگر برای خواندن پایدار باشد.

```

/* USER CODE BEGIN WHILE */
while (1)
{
    uint32_t sum_adc = 0;
    uint8_t valid_samples = 0;

    // 1. Force the STM32 to wait for EXACTLY 50 successful conversions
    while(valid_samples < 50)
    {
        HAL_ADC_Start(&hadc1);
    }
}

```

```

        if(HAL_ADC_PollForConversion(&hadc1, HAL_MAX_DELAY) == HAL_OK)
        {
            sum_adc += HAL_ADC_GetValue(&hadc1);
            valid_samples++;
        }
    }

    // Calculate the true average
    uint16_t avg_adc = sum_adc / 50;

    // Multiply by 330 (instead of 3.3) to keep everything as whole numbers
    uint32_t volt_x100 = (avg_adc * 330) / 4095;

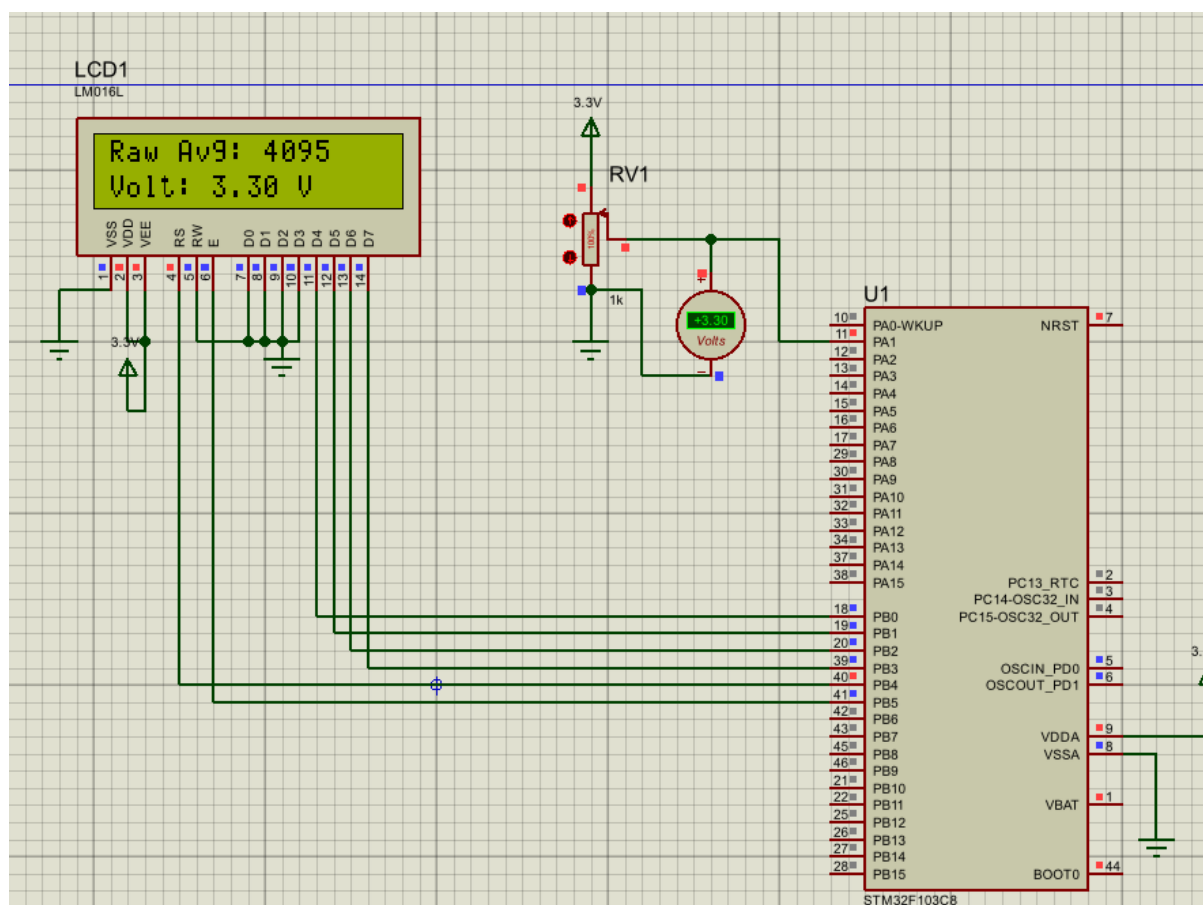
    uint8_t whole_part = volt_x100 / 100;    // Extracts the '3'
    uint8_t decimal_part = volt_x100 % 100;  // Extracts the '30'

    // Print the Raw integer average to Row 0
    snprintf(lcd_buffer, sizeof(lcd_buffer), "Raw Avg: %d    ", avg_adc);
    Lcd_cursor(&lcd, 0, 0);
    Lcd_string(&lcd, lcd_buffer);

    // Print the Voltage using our safe integers (e.g., "Volt: 3.30 V")
    // %d prints the whole number, %02d ensures the decimal prints two
digits (like .05)
    snprintf(lcd_buffer, sizeof(lcd_buffer), "Volt: %d.%02d V    ",
whole_part, decimal_part);
    Lcd_cursor(&lcd, 1, 0);
    Lcd_string(&lcd, lcd_buffer);

    HAL_Delay(500);
/* USER CODE END WHILE */

```



میکروکنترلر STM32F103C8 (چون کنترلر اصلی ما در داخل Proteus موجود نبود) رو در محیط شبیه سازی قرار میدهم

LCD LM016L را در محیط Proteus قرار می دهیم وپین ان را مطابق شکل به میکروکنترلر وصل می کنیم.

پتانسیومتر POT-HG را در محیط Proteus قرار میدهم و ان را به صورت تصویر بالا پیکر بندی می کنیم.

در اخر هم فایل Compile شده HEX. رو به کنترلر می دهیم.

4. بخش MATLAB و Virtual serial port pro

این بخش برای حل این سوال لازم نمی باشد.

ویدیوی شبیه سازی در داخل پوشه تمرین موجود می باشد.

1.2. روش interrupt

در روش اینترپت، میکروکنترلر بی‌آنکه مدام چک کند (پولینگ)، فقط وقتی داده می‌رسد وقفه ایجاد می‌شود و CPU در بقیه زمان آزاد است و درگیر نمی‌شود.

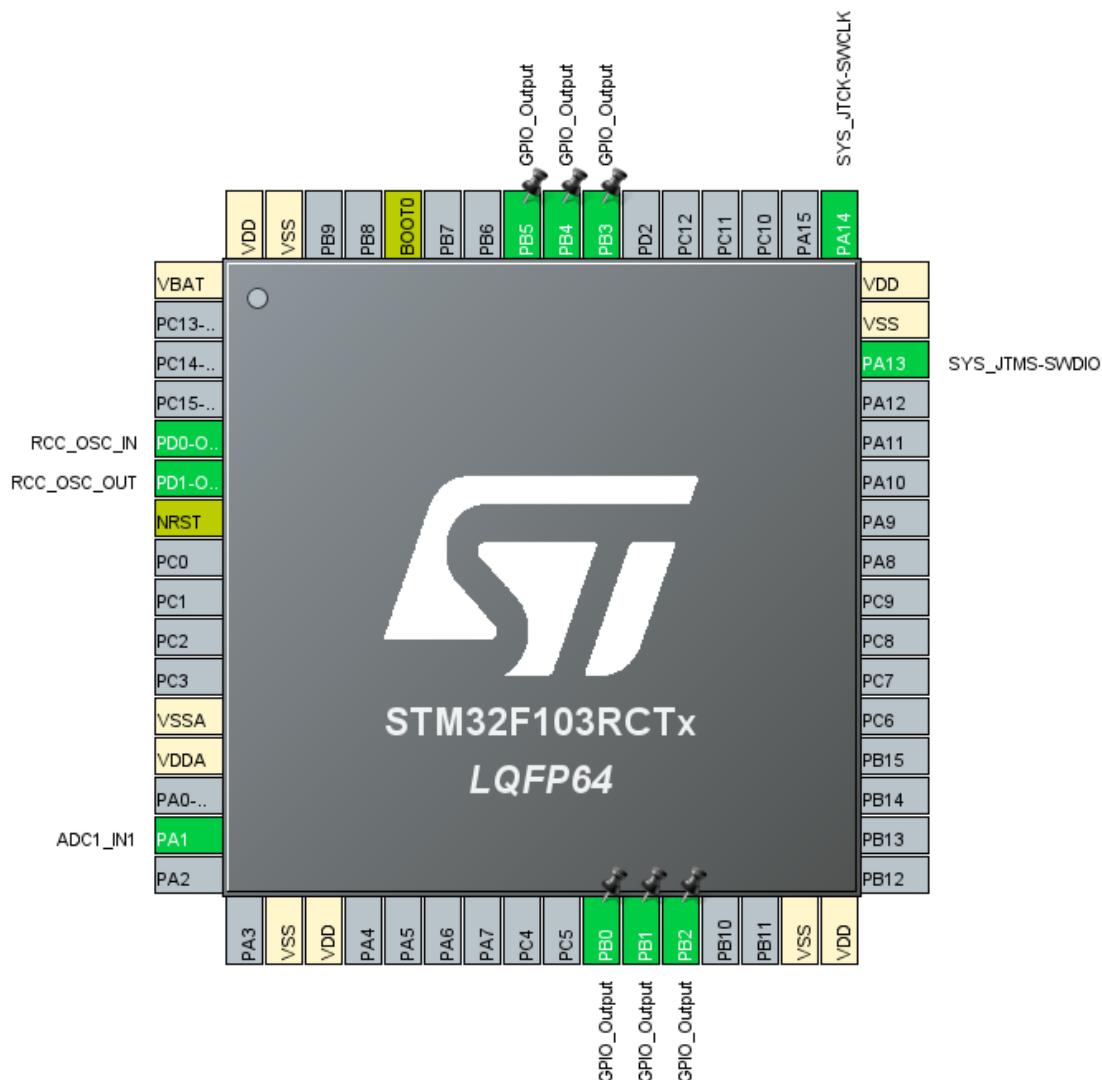
بخش STM32CubeMX

میکرو کنترلر STM32F103RCT رو در نرم افزار انتخاب میکنیم.

تیک مربوط به وقفه را می‌زنیم

پروگرامر رو به حالت Serial Wire می‌کنیم

STM32 رو مطابق تصاویر زیر پیکر بندی میکنیم.



ADC1 Mode and Configuration

Mode

☐ IN0
☒ IN1
☐ IN2

Configuration

Reset Configuration

✓ Parameter Settings
✓ User Constants
✓ NVIC Settings
✓ DMA Settings
✓ GPIO Settings

Configure the below parameters :

⏪ ⏩

▼ ADCs_Common_Settings
 ▼ ADC_Settings
 ▼ ADC_Regular_ConversionMode
 ▼
 ▼ ADC_Injected_ConversionMode
 ▼ WatchDog

Mode	Independent mode
Data Alignment	Right alignment
Scan Conversion Mode	Disabled
Continuous Conversion Mode	Disabled
Discontinuous Conversion Mode	Disabled
Enable Regular Conversions	Enable
Number Of Conversion	1
External Trigger Conversion Source	Regular Conversion launched by software
Rank	1
Channel	Channel 1
Sampling Time	239.5 Cycles
Enable Injected Conversions	Disable
Enable Analog WatchDog Mode	☐

ADC1 Mode and Configuration			
Mode			
☐ IN0 ☒ IN1 ☐ IN2			
Configuration			
Reset Configuration			
✓ Parameter Settings ✓ User Constants ✓ NVIC Settings ✓ DMA Settings ✓ GPIO Settings			
NVIC Interrupt Table	Enabled	Preemption Priority	Sub Priority
ADC1 and ADC2 global interrupts	✓	0	0

در این بخش فایل هدر lcd.h به پروژه اضافه شده است. این فایل برای استفاده از توابع کتابخانه‌ی LCD مورد نیاز است.

```
/* USER CODE BEGIN Includes */
#include "lcd.h"
/* USER CODE END Includes */
```

در این بخش یک هندل برای LCD به نام lcd، یک بافر ۱۶ کاراکتری برای نمایش روی LCD، و چند متغیر برای استفاده در روال وقفه‌ای ADC تعریف شده است. متغیرهای adc_sum، sample_count و data_ready_flag با کلمه‌ی کلیدی volatile مشخص شده‌اند تا تغییرات آن‌ها درون وقفه به‌درستی در حلقه‌ی اصلی دیده شود. همچنین متغیر avg_adc برای نگهداری میانگین محاسبه‌شده در نظر گرفته شده است.

```
/* USER CODE BEGIN PV */
Lcd_HandleTypeDef lcd;
char lcd_buffer[16];
// Variables marked volatile because they are shared/modified inside the
Interrupt ISR
volatile uint32_t adc_sum = 0;
volatile uint16_t sample_count = 0;
volatile uint8_t data_ready_flag = 0;
uint16_t avg_adc;
/* USER CODE END PV */
```

در این بخش تابع کال‌بک HAL_ADC_ConvCpltCallback پیاده‌سازی شده است. این تابع پس از کامل‌شدن هر تبدیل ADC به‌صورت وقفه‌ای فراخوانی می‌شود. درون آن مقدار خوانده‌شده از ADC با adc_sum جمع و شمارنده‌ی sample_count یک واحد افزایش می‌یابد. وقتی شمارنده به ۵۰ برسد، پرچم data_ready_flag برابر ۱ می‌شود تا حلقه‌ی اصلی متوجه آماده‌بودن داده‌ها شود. در غیر این صورت یک وقفه‌ی جدید برای ادامه‌ی نمونه‌برداری درخواست می‌شود.

```

/* USER CODE BEGIN 0 */
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc)
{
    if(hadc->Instance == ADC1)
    {
        adc_sum += HAL_ADC_GetValue(hadc);
        sample_count++;

        if (sample_count >= 50)
        {
            data_ready_flag = 1; // Signal the main loop that the batch is
ready
        }
        else
        {
            HAL_ADC_Start_IT(hadc); // Fire the next interrupt conversion
        }
    }
}
/* USER CODE END 0 */

```

در این بخش LCD با استفاده از پورت‌ها و پین‌های مشخص شده راه‌اندازی می‌شود. سپس یک پیام اولیه («ADC IT Mode») روی نمایشگر نوشته می‌شود و با یک ثانیه تأخیر و پاک کردن صفحه، زنجیره‌ی وقفه‌ای ADC با فراخوانی HAL_ADC_Start_IT آغاز می‌گردد.

```

/* USER CODE BEGIN 2 */
Lcd_PortType ports[] = { GPIOB, GPIOB, GPIOB, GPIOB };
Lcd_PinType pins[] = {GPIO_PIN_0, GPIO_PIN_1, GPIO_PIN_2, GPIO_PIN_3};
lcd = Lcd_create(ports, pins, GPIOB, GPIO_PIN_4, GPIOB, GPIO_PIN_5,
LCD_4_BIT_MODE);
Lcd_cursor(&lcd, 0, 0);
Lcd_string(&lcd, "ADC IT Mode");
HAL_Delay(1000);
Lcd_clear(&lcd);
HAL_ADC_Start_IT(&hadc1);
/* USER CODE END 2 */

```

حلقه‌ی اصلی برنامه در این بخش زمانی که پرچم data_ready_flag فعال شود، میانگین ۵۰ نمونه‌ی ADC را محاسبه می‌کند. سپس با استفاده از محاسبات عدد صحیح (ضرب میانگین در ۳۳۰ و تقسیم بر ۴۰۹۵) ولتاژ به دست می‌آید و بخش صحیح و اعشاری آن جدا می‌شود. مقادیر میانگین خام و ولتاژ

محاسبه‌شده روی دو سطر LCD نمایش داده می‌شوند. پس از نمایش، متغیرهای جمع‌کننده و پرچم ریست می‌شوند و با ۵۰۰ میلی‌ثانیه تأخیر، یک وقفه‌ی جدید برای دریافت ۵۰ نمونه‌ی بعدی درخواست می‌شود.

```
/* USER CODE BEGIN WHILE */
while (1)
{
    if (data_ready_flag == 1)
    {
        // Calculate average using integer division
        avg_adc = adc_sum / 50;

        // Integer-based scaling factor matching (avg_adc * 330) / 4095
        uint32_t volt_scaled = (avg_adc * 330) / 4095;
        uint8_t whole_part = volt_scaled / 100;
        uint8_t frac_part = volt_scaled % 100;

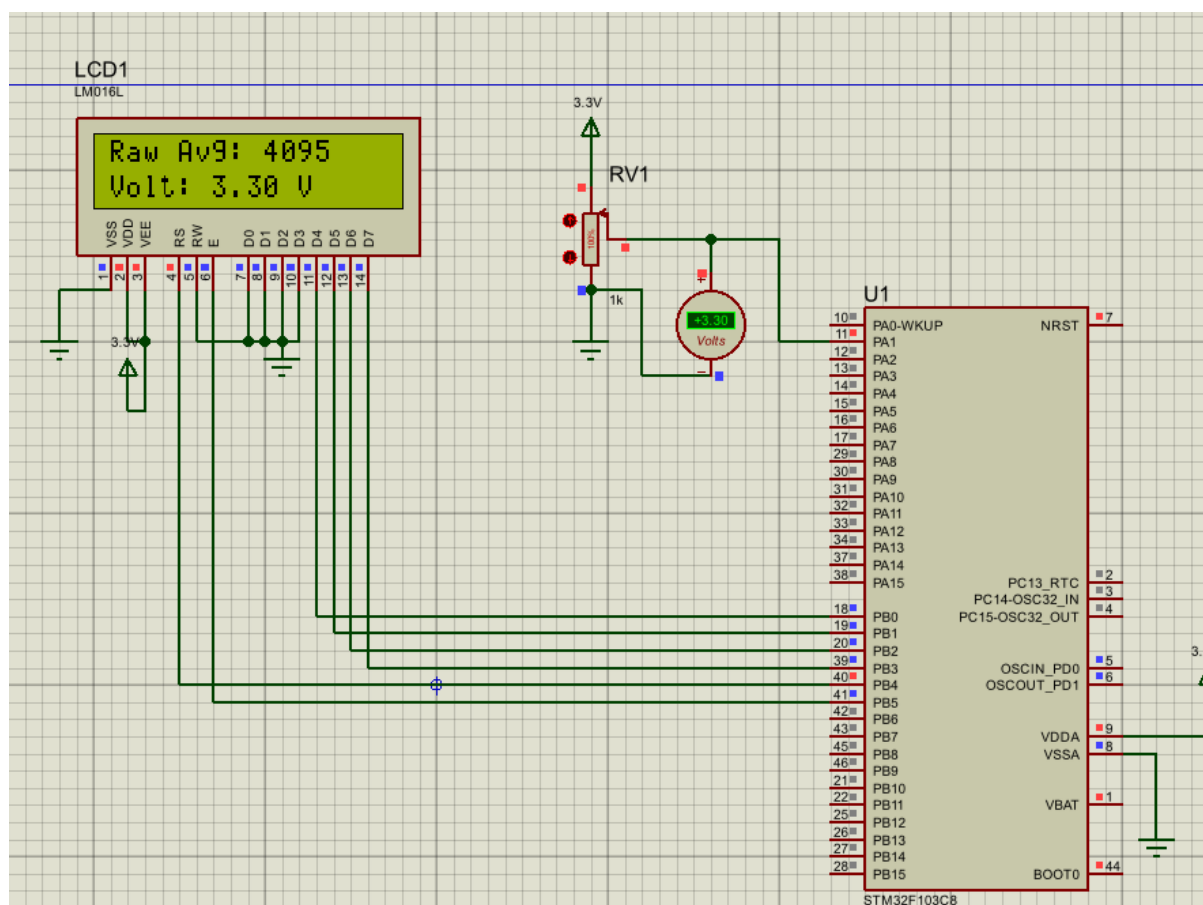
        // Print Raw value on line 0
        snprintf(lcd_buffer, sizeof(lcd_buffer), "Raw Avg: %d    ", avg_adc);
        Lcd_cursor(&lcd, 0, 0);
        Lcd_string(&lcd, lcd_buffer);

        // Print Voltage calculation on line 1
        snprintf(lcd_buffer, sizeof(lcd_buffer), "Volt: %d.%02d V    ",
whole_part, frac_part);
        Lcd_cursor(&lcd, 1, 0);
        Lcd_string(&lcd, lcd_buffer);

        // Clear flags and state variables safely
        adc_sum = 0;
        sample_count = 0;
        data_ready_flag = 0;

        HAL_Delay(500); // UI update delay

        // Re-enable the interrupt chain for the next window
        HAL_ADC_Start_IT(&hadc1);
    }
}
/* USER CODE END WHILE */
```



میکروکنترلر STM32F103C8 (چون کنترلر اصلی ما در داخل Proteus موجود نبود) رو در محیط

شبیه سازی قرار میدهیم

LCD LM016L را در محیط Proteus قرار می دهیم وپین ان را مطابق شکل به میکروکنترلر وصل می کنیم.

پتانسیومتر POT-HG را در محیط Proteus قرار میدهیم و ان را به صورت تصویر بالا پیکر بندی می کنیم.

در اخر هم فایل Compile شده HEX. رو به کنترلر می دهیم.

4. بخش MATLAB و Virtual serial port pro

برای حل این سوال لازم نمی باشد

ویدیوی شبیه سازی در داخل پوشه تمرین موجود می باشد.

1.3. روش DMA

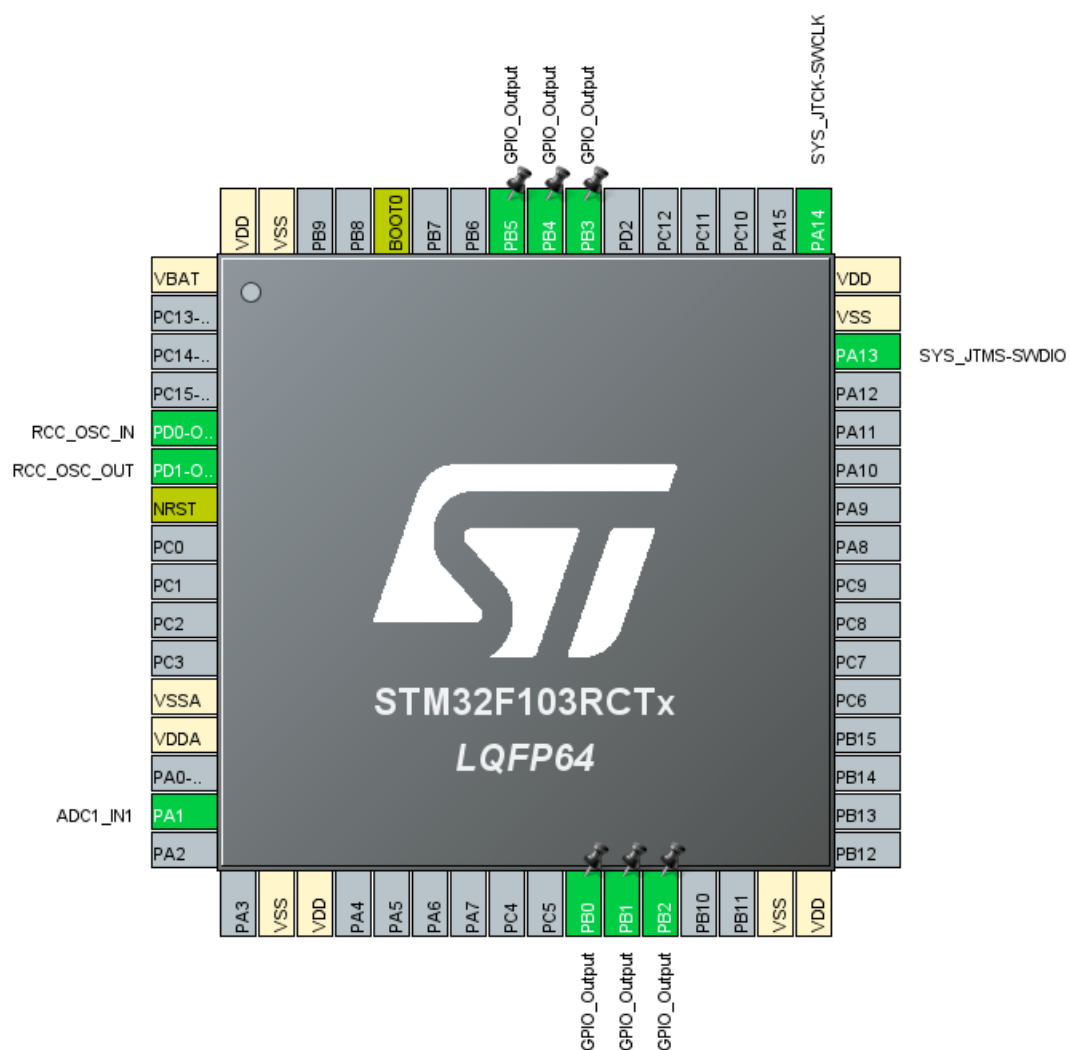
در روش DMA، میکروکنترلر بی‌آنکه مدام در انتقال داده دخالت کند (بایت به بایت)، فقط در ابتدا تنظیمات اولیه را انجام می‌دهد و سپس کنترلر DMA مستقیماً داده را جابه‌جا می‌کند و CPU در بقیه زمان آزاد است و درگیر نمی‌شود.

(بخش *STM32CubeMX*)

میکروکنترلر STM32F103RCT رو در نرم افزار انتخاب میکنیم.

پروگرامر رو به حالت Serial Wire و کلاک رو تنظیم میکنیم

STM32 رو مطابق تصاویر زیر پیکر بندی میکنیم.



ADC1 Mode and Configuration

Mode

☐ IN0
☒ IN1
☐ IN2

Configuration

Reset Configuration

☒ Parameter Settings
 ☒ User Constants
 ☒ NVIC Settings
 ☒ DMA Settings
 ☒ GPIO Settings

Configure the below parameters :

ADCs_Common_Settings

Mode

Independent mode

ADC_Settings

Data Alignment

Right alignment

Scan Conversion Mode

Disabled

Continuous Conversion Mode

Enabled

Discontinuous Conversion Mode

Disabled

ADC_Regular_ConversionMode

Enable Regular Conversions

Enable

Number Of Conversion

1

External Trigger Conversion Source

Regular Conversion launched by software

Rank

1

Channel

Channel 1

Sampling Time

239.5 Cycles

ADC_Injected_ConversionMode

Enable Injected Conversions

Disable

WatchDog

Enable Analog WatchDog Mode

☐

ADC1 Mode and Configuration

Mode

☐ IN0
☒ IN1
☐ IN2

Configuration

Reset Configuration

☒ Parameter Settings
 ☒ User Constants
 ☒ NVIC Settings
 ☒ DMA Settings
 ☒ GPIO Settings

DMA Request	Channel	Direction	Priority
ADC1	DMA1 Channel 1	ADC1	Low

در این قسمت فایل‌های هدر <stdio.h> و lcd.h به پروژه اضافه شده‌اند. stdio.h برای استفاده از تابع snprintf و lcd.h برای کار با کتابخانه‌ی LCD لازم است.

```
/* USER CODE BEGIN Includes */
#include <stdio.h>
#include "lcd.h"
/* USER CODE END Includes */
```

در این بخش یک ثابت پیش‌پردازنده به نام ADC_SAMPLE_COUNT با مقدار ۵۰ تعریف شده است. از این ثابت برای مشخص کردن تعداد نمونه‌هایی که DMA باید جمع‌آوری کند استفاده می‌شود.

```
/* USER CODE BEGIN PD */
#define ADC_SAMPLE_COUNT 50
/* USER CODE END PD */
```

چندین متغیر سراسری برای پروژه تعریف شده‌اند: یک هندل LCD به نام lcd، یک بافر ۱۶ کاراکتری به نام lcd_buffer، یک آرایه‌ی ۵۰ تایی از اعداد ۱۶ بیتی بدون علامت (adc_values) که توسط DMA به‌طور خودکار پر می‌شود، یک پرچم data_ready_flag که در وقفه‌ی DMA فعال می‌شود، و دو متغیر adc_sum و adc_avg برای محاسبات متغیرهای adc_values و data_ready_flag با volatile تعریف شده‌اند تا تغییرات آن‌ها در وقفه‌ها به‌درستی در حلقه‌ی اصلی منعکس شود.

```
/* USER CODE BEGIN PV */
Lcd_HandleTypeDef lcd;
char lcd_buffer[16];

// 50-sample buffer filled automatically by DMA hardware
volatile uint16_t adc_values[ADC_SAMPLE_COUNT];
volatile uint8_t data_ready_flag = 0;

uint32_t adc_sum = 0;
uint16_t adc_avg = 0;
/* USER CODE END PV */
```

راه‌اندازی اولیه‌ی LCD در این بخش انجام می‌شود: پورت‌ها و پین‌های داده و کنترل مشخص شده و با Lcd_create یک هندل LCD ساخته می‌شود. سپس یک پیام اولیه («ADC Circular DMA») روی LCD نوشته می‌شود و پس از یک ثانیه مکث، صفحه پاک می‌شود. در نهایت DMA دایره‌ای و پیوسته‌ی ADC تنها یک‌بار با دستور HAL_ADC_Start_DMA آغاز می‌شود؛ به دلیل حالت دایره‌ای، نیازی به فراخوانی دوباره نیست.

```
/* USER CODE BEGIN 2 */
Lcd_PortType dataPorts[] = {GPIOB, GPIOB, GPIOB, GPIOB};
Lcd_PinType dataPins[] = {GPIO_PIN_0, GPIO_PIN_1, GPIO_PIN_2, GPIO_PIN_3};
lcd = Lcd_create(dataPorts, dataPins, GPIOB, GPIO_PIN_4, GPIOB, GPIO_PIN_5,
LCD_4_BIT_MODE);
```

```

Lcd_cursor(&lcd, 0, 0);
Lcd_string(&lcd, "ADC Circular DMA");
HAL_Delay(1000);
Lcd_clear(&lcd);

// Start DMA ONCE. Because it's circular + continuous, you NEVER have to
call this again.
HAL_ADC_Start_DMA(&hadc1, (uint32_t*)adc_values, ADC_SAMPLE_COUNT);
/* USER CODE END 2 */

```

حلقه‌ی اصلی برنامه منتظر فعال شدن پرچم `data_ready_flag` می‌ماند. وقتی این پرچم بر اثر تکمیل ۵۰ تبدیل ADC توسط DMA فعال شود، پرچم بلافاصله صفر می‌شود تا از پردازش مجدد جلوگیری شود. سپس تمام ۵۰ نمونه‌ی موجود در آرایه‌ی `adc_values` جمع زده شده و میانگین آن‌ها محاسبه می‌شود. با استفاده از عملیات ریاضی با اعداد صحیح، ولتاژ معادل (با دو رقم اعشار) به دست می‌آید. در ادامه مقدار میانگین خام و ولتاژ محاسبه شده در دو سطر LCD نمایش داده می‌شود و یک تأخیر ۵۰۰ میلی ثانیه‌ای برای خوانایی نمایشگر اعمال می‌شود. لازم به یادآوری است که در تمام این مدت سخت افزار DMA مشغول پرکردن دوباره‌ی آرایه است و برنامه هیچ گاه متوقف نمی‌شود.

```

/* USER CODE BEGIN WHILE */
while (1)
{
    if (data_ready_flag == 1)
    {
        // 1) Clear flag immediately
        data_ready_flag = 0;

        // 2) Sum up the 50 samples (the hardware is already collecting the
next batch!)
        adc_sum = 0;
        for (int i = 0; i < ADC_SAMPLE_COUNT; i++)
        {
            adc_sum += adc_values[i];
        }
        adc_avg = adc_sum / ADC_SAMPLE_COUNT;

        // 3) Integer voltage math
        uint32_t volt_scaled = (adc_avg * 330) / 4095;
        uint8_t whole_part = volt_scaled / 100;
        uint8_t frac_part = volt_scaled % 100;

        // 4) Display data
        snprintf(lcd_buffer, sizeof(lcd_buffer), "Raw Avg: %d  ", adc_avg);
        Lcd_cursor(&lcd, 0, 0);
    }
}
/* USER CODE END WHILE */

```

```

        Lcd_string(&lcd, lcd_buffer);

        snprintf(lcd_buffer, sizeof(lcd_buffer), "Volt: %d.%02d V  ",
whole_part, frac_part);
        Lcd_cursor(&lcd, 1, 0);
        Lcd_string(&lcd, lcd_buffer);

        // 5) Delay just to give the screen breathing room (keeps UI readable)
        HAL_Delay(500);
    }
    /* USER CODE END WHILE */

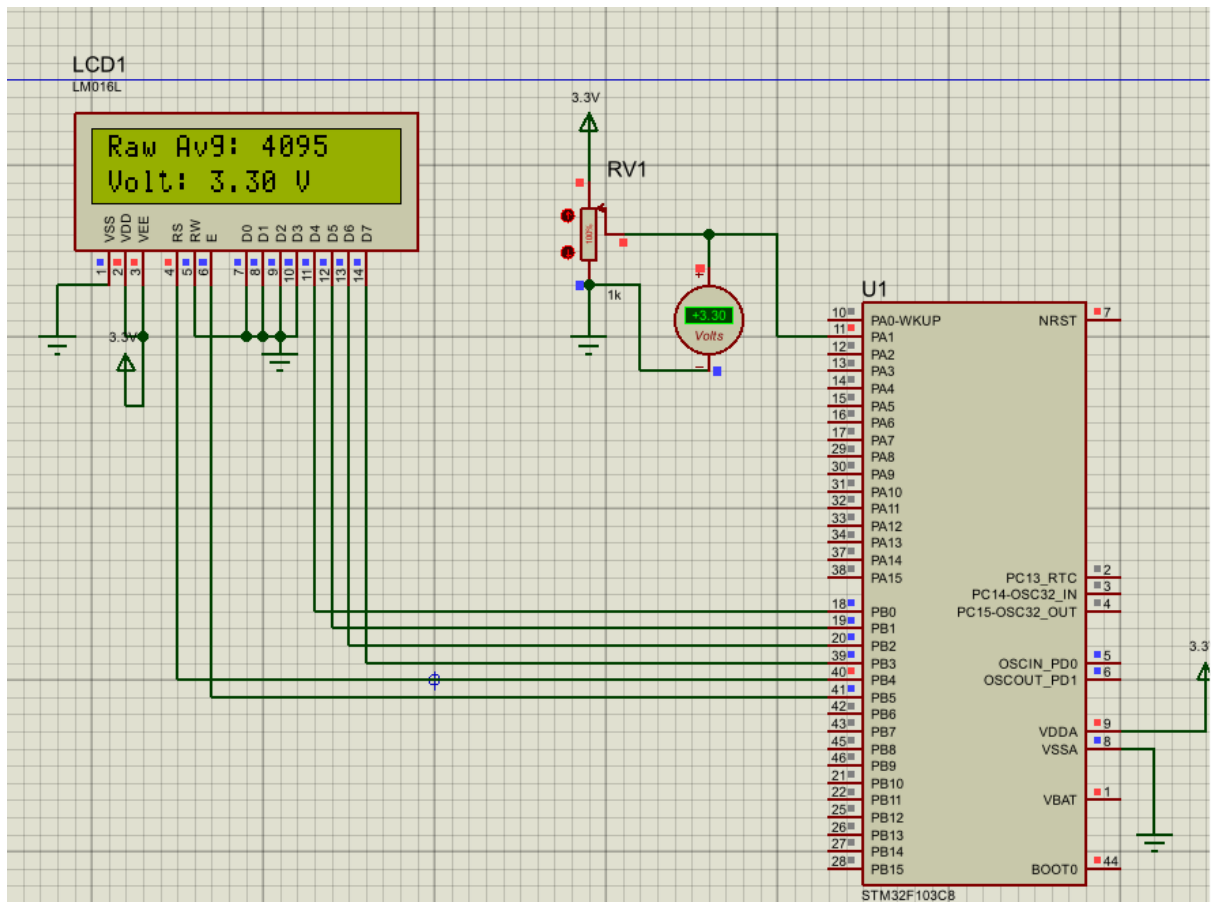
```

در این بخش تابع کال‌بک HAL_ADC_ConvCpltCallback پیاده‌سازی شده است. این تابع توسط کتابخانه‌ی HAL پس از کامل شدن تبدیل ADC (در واقع سرریز بافر DMA در حالت دایره‌ای) فراخوانی می‌شود. وقتی نمونه‌برداری از ADC1 کامل شود، پرچم data_ready_flag برابر ۱ قرار می‌گیرد تا حلقه‌ی اصلی مطلع شود که می‌تواند یک بسته‌ی ۵۰ تایی جدید را پردازش کند.

```

/* USER CODE BEGIN 4 */
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc)
{
    if(hadc->Instance == ADC1)
    {
        data_ready_flag = 1;
    }
}
/* USER CODE END 4 */

```



میکروکنترلر STM32F103C8 (چون کنترلر اصلی ما در داخل Proteus موجود نبود) رو در محیط شبیه سازی قرار میدهم

LCD LM016L را در محیط Proteus قرار می دهیم وپین ان را مطابق شکل به میکروکنترلر وصل می کنیم.

پتانسیومتر POT-HG را در محیط Proteus قرار میدهم و ان را به صورت تصویر بالا پیکر بندی می کنیم.

در اخر هم فایل Compile شده HEX. رو به کنترلر می دهیم.

4. بخش Matlab و Virtual serial port pro

این بخش برای حل این سوال لازم نیست.

ویدیوی شبیه سازی در داخل پوشه تمرین موجود می باشد.

2. تمرین فوق را برای پایه های ADC1_IN1 و ADC1_IN2 و ADC1_IN3، تکرار کنید. در این تمرین، بجای استفاده از چند پتانسیومتر، می توانید فقط یک پتانسیومتر در بُرد داشته باشید و خروجی آن را، هر بار به یکی از پایه های فوق، وصل نمایید.

2.1. روش Poling

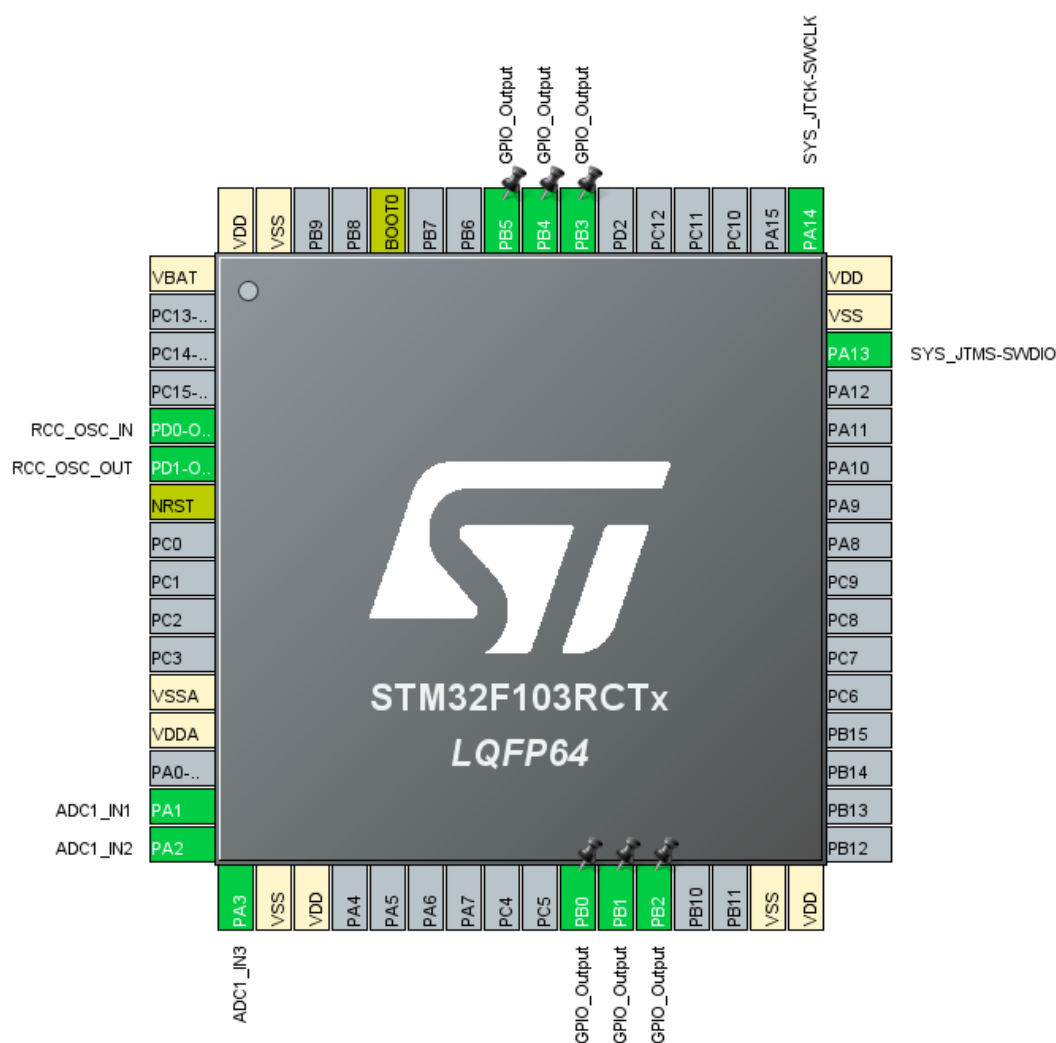
در این روش CPU به طور مداوم وضعیت را بررسی می کند.

(بخش *STM32CubeMX*)

میکرو کنترلر STM32F103RCT رو در نرم افزار انتخاب میکنیم.

پروگرامر رو به حالت Serial Wire و کلاک رو تنظیم میکنیم

STM32 رو مطابق تصاویر زیر پیکر بندی میکنیم.



ADC1 Mode and Configuration

Mode

☐ IN0
☒ IN1
☒ IN2
☒ IN3

Configuration

Reset Configuration

☒ Parameter Settings
 ☒ User Constants
 ☒ NVIC Settings
 ☒ DMA Settings
 ☒ GPIO Settings

Configure the below parameters :

ADCs_Common_Settings

Mode

Independent mode

ADC_Settings

Data Alignment

Right alignment

Scan Conversion Mode

Disabled

Continuous Conversion Mode

Disabled

Discontinuous Conversion Mode

Disabled

ADC_Regular_ConversionMode

Enable Regular Conversions

Enable

Number Of Conversion

1

External Trigger Conversion Source

Regular Conversion launched by software

Rank

1

Channel

Channel 1

Sampling Time

239.5 Cycles

ADC_Injected_ConversionMode

Enable Injected Conversions

Disable

WatchDog

Enable Analog WatchDog Mode

☐

در اینجا فایل‌های سرآیند lcd.h و <stdio.h> به پروژه افزوده شده‌اند. lcd.h برای ارتباط با نمایشگر کاراکتری و <stdio.h> برای استفاده از تابع sprintf به کار می‌رود.

```
/* USER CODE BEGIN Includes */
#include "lcd.h"
#include <stdio.h>
/* USER CODE END Includes */
```

در این بخش یک تعریف پیش‌پردازنده به نام ADC_SAMPLE_COUNT با مقدار ۵۰ قرار داده شده است. از این ثابت به عنوان تعداد نمونه‌های هر کانال در میانگین‌گیری نرم‌افزاری استفاده می‌شود.

```
/* USER CODE BEGIN PD */
#define ADC_SAMPLE_COUNT 50 // Number of samples for software averaging
/* USER CODE END PD */
```

دو متغیر سراسری تعریف شده‌اند: یک هندل برای LCD با نام lcd و یک بافر ۳۲ کاراکتری به نام lcd_buffer که برای ذخیره‌سازی رشته‌های نمایشی پیش از ارسال به LCD استفاده خواهد شد.

```
Lcd_HandleTypeDef lcd;
char lcd_buffer[32];
/* USER CODE END PV */
```

نمونه‌اولیه تابع Measure_And_Display_ADC_Polling در اینجا اعلان شده است. این تابع وظیفه اندازه‌گیری یک کانال ADC به روش polling و سپس نمایش میانگین و ولتاژ روی LCD را بر عهده دارد.

```
/* USER CODE BEGIN PFP */
void Measure_And_Display_ADC_Polling(uint32_t target_channel, const char*
channel_name);
/* USER CODE END PFP */
```

بدنه تابع Measure_And_Display_ADC_Polling در این بخش پیاده‌سازی شده است. ابتدا رجیستر ترتیب کانال (SQR3) ریست می‌شود تا از تداخل کانال‌ها جلوگیری گردد. سپس کانال درخواستی با زمان نمونه‌برداری مشخص پیکربندی شده و یک تأخیر ۵ میلی‌ثانیه‌ای برای تثبیت مالتی‌پلکسر آنالوگ در نظر گرفته می‌شود. یک خواندن ساختگی (dummy read) انجام می‌شود تا لوله‌کشی داخلی ADC کاملاً روی کانال جدید قفل شود. پس از آن ۵۰ نمونه به صورت تکی و با شروع/توقف ADC خوانده و جمع می‌شوند. میانگین این نمونه‌ها محاسبه شده و ولتاژ بر حسب میلی‌ولت (حاصل ضرب میانگین در ۳۳۰۰ و تقسیم بر ۴۰۹۵) به دست می‌آید. بخش صحیح و دو رقم اعشار ولتاژ استخراج شده و همراه با نام کانال در سطر اول LCD نمایش داده می‌شود. همچنین مقدار خام میانگین با عرض ثابت ۴ کاراکتر در سطر دوم نشان داده می‌شود. در پایان یک ثانیه تأخیر برای خوانایی پیش از تغییر به کانال بعدی اعمال می‌گردد.

```
/* USER CODE BEGIN 0 */
void Measure_And_Display_ADC_Polling(uint32_t target_channel, const char*
channel_name)
{
    ADC_ChannelConfTypeDef adc_config = {0};
    uint32_t accumulated_samples = 0;
    uint16_t average_adc_value = 0;
```

```

// Reset the internal routing mux to prevent cross-channel ghosting
hadc1.Instance->SQR3 = 0;

// Configure the active ADC channel
adc_config.Channel = target_channel;
adc_config.Rank = ADC_REGULAR_RANK_1;
adc_config.SamplingTime = ADC_SAMPLETIME_239CYCLES_5;
HAL_ADC_ConfigChannel(&hadc1, &adc_config);

// Hardware stabilization delay for analog multiplexer
HAL_Delay(5);

// --- PIPELINE FLUSH (Dummy Read) ---
// Forces the ADC to latch onto the new channel properly
HAL_ADC_Start(&hadc1);
HAL_ADC_PollForConversion(&hadc1, HAL_MAX_DELAY);
HAL_ADC_GetValue(&hadc1);
HAL_ADC_Stop(&hadc1);

// Acquire samples for averaging
for (uint8_t sample_idx = 0; sample_idx < ADC_SAMPLE_COUNT; sample_idx++)
{
    HAL_ADC_Start(&hadc1);
    HAL_ADC_PollForConversion(&hadc1, HAL_MAX_DELAY);
    accumulated_samples += HAL_ADC_GetValue(&hadc1);
    HAL_ADC_Stop(&hadc1);
}

// Compute average
average_adc_value = accumulated_samples / ADC_SAMPLE_COUNT;

// High-precision voltage calculation in millivolts (3.3V -> 3300mV)
uint32_t voltage_mv = (average_adc_value * 3300) / 4095;
uint8_t display_volts = voltage_mv / 1000;
uint8_t display_frac = (voltage_mv % 1000) / 10; // Extract 2 decimal
places

// Update LCD without Lcd_clear() to eliminate flickering.
// The trailing spaces overwrite leftover characters from previous
strings.
snprintf(lcd_buffer, sizeof(lcd_buffer), "%s Volt: %d.%02d V ",
channel_name, display_volts, display_frac);
Lcd_cursor(&lcd, 0, 0);
Lcd_string(&lcd, lcd_buffer);

// Use %4u to ensure the raw ADC value always takes up 4 character spaces

```

```

    snprintf(lcd_buffer, sizeof(lcd_buffer), "%s Mean: %4u   ", channel_name,
average_adc_value);
    Lcd_cursor(&lcd, 1, 0);
    Lcd_string(&lcd, lcd_buffer);

    // Hold display for 1 second before multiplexing to the next channel
    HAL_Delay(1000);
}
/* USER CODE END 0 */

```

در این بخش راه اندازی اولیه LCD انجام می‌شود: پورت‌ها و پایه‌های داده و کنترل مشخص شده و یک هندل LCD ایجاد می‌شود. سپس تنظیمات پین‌های PA1, PA2, PA3 به صورت آنالوگ و بدون پول‌آپ/پول‌داون باز نویسی می‌شود تا مطمئن شویم این پایه‌ها به عنوان ورودی آنالوگ باقی می‌مانند و از خواندن نویز دیجیتال جلوگیری شود. در نهایت یک کالیبراسیون ADC با فراخوانی HAL_ADCEX_Calibration_Start اجرا می‌شود تا دقت اندازه‌گیری افزایش یابد.

```

/* USER CODE BEGIN 2 */
Lcd_PortType ports[] = { GPIOB, GPIOB, GPIOB, GPIOB };
Lcd_PinType pins[] = {GPIO_PIN_0, GPIO_PIN_1, GPIO_PIN_2, GPIO_PIN_3};
lcd = Lcd_create(ports, pins, GPIOB, GPIO_PIN_4, GPIOB, GPIO_PIN_5,
LCD_4_BIT_MODE);
// 2. THE ANALOG OVERRIDE
// Executing this after MX_ADC1_Init() guarantees the pins stay analog
// and prevents the multiplexer from reading digital floating noise.
GPIO_InitTypeDef GPIO_InitStruct = {0};
GPIO_InitStruct.Pin = GPIO_PIN_1 | GPIO_PIN_2 | GPIO_PIN_3;
GPIO_InitStruct.Mode = GPIO_MODE_ANALOG;
GPIO_InitStruct.Pull = GPIO_NOPULL;
HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

// 3. Calibrate ADC
HAL_ADCEX_Calibration_Start(&hadc1);
/* USER CODE END 2 */

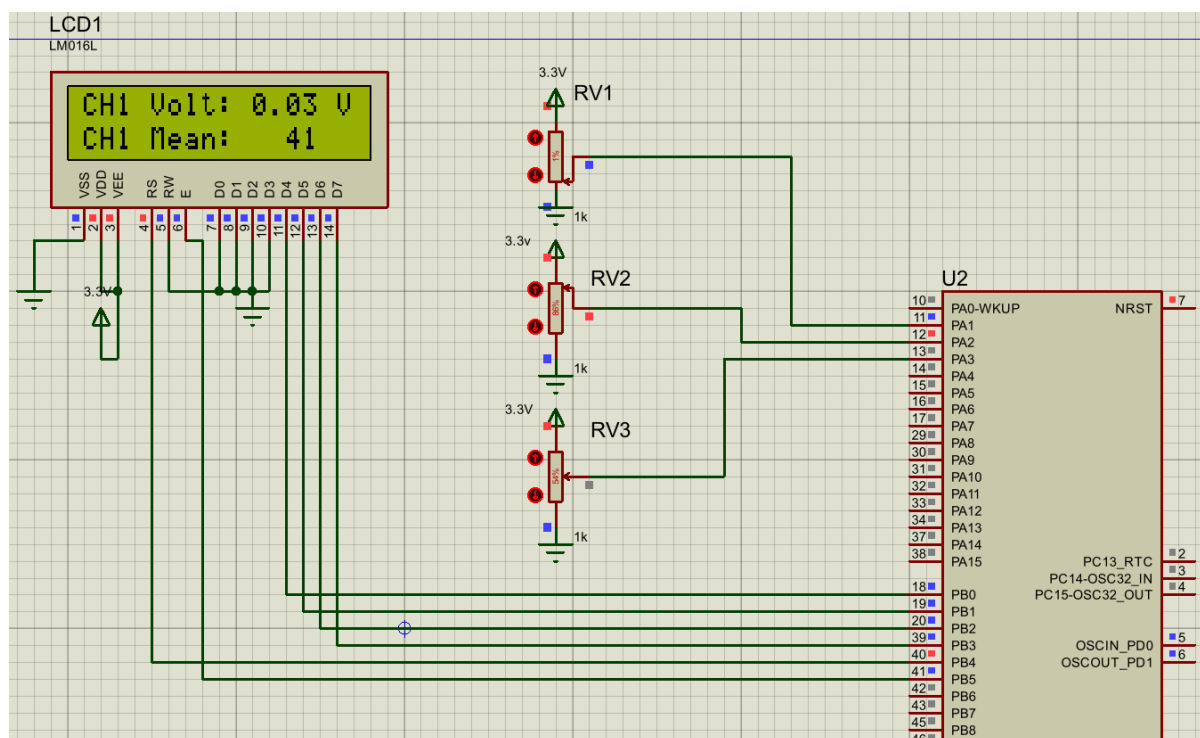
```

بدنه حلقه بی‌نهایت برنامه در اینجا نوشته شده است. سه کانال ADC به ترتیب و با فراخوانی تابع Measure_And_Display_ADC_Polling نمونه‌برداری و نمایش داده می‌شوند: ابتدا کانال ۱ با برچسب "CH1"، سپس کانال ۲ با "CH2" و در نهایت کانال ۳ با "CH3". این چرخه به‌طور مداوم تکرار می‌شود.

```

/* USER CODE BEGIN 3 */
// Sequentially sample and display each channel
Measure_And_Display_ADC_Polling(ADC_CHANNEL_1, "CH1");
Measure_And_Display_ADC_Polling(ADC_CHANNEL_2, "CH2");
Measure_And_Display_ADC_Polling(ADC_CHANNEL_3, "CH3");
}
/* USER CODE END 3 */

```



میکروکنترلر STM32F103C8 (چون کنترلر اصلی ما در داخل Proteus موجود نبود) رو در محیط شبیه سازی قرار میدهم

LCD LM016L را در محیط Proteus قرار می دهیم وپین ان را مطابق شکل به میکروکنترلر وصل می کنیم.

پتانسیومتر POT-HG را در محیط Proteus قرار میدهم و ان را به صورت تصویر بالا پیکر بندی می کنیم.

در اخر هم فایل Compile شده HEX. رو به کنترلر می دهیم.

4. بخش MATLAB و Virtual serial port pro

این بخش برای حل این سوال لازم نمی باشد.

ویدیوی شبیه سازی در داخل پوشه تمرین موجود می باشد.

2.2. روش interrupt

در روش اینترپت، میکروکنترلر بی‌آنکه مدام چک کند (پولینگ)، فقط وقتی داده می‌رسد وقفه ایجاد می‌شود و CPU در بقیه زمان آزاد است و درگیر نمی‌شود.

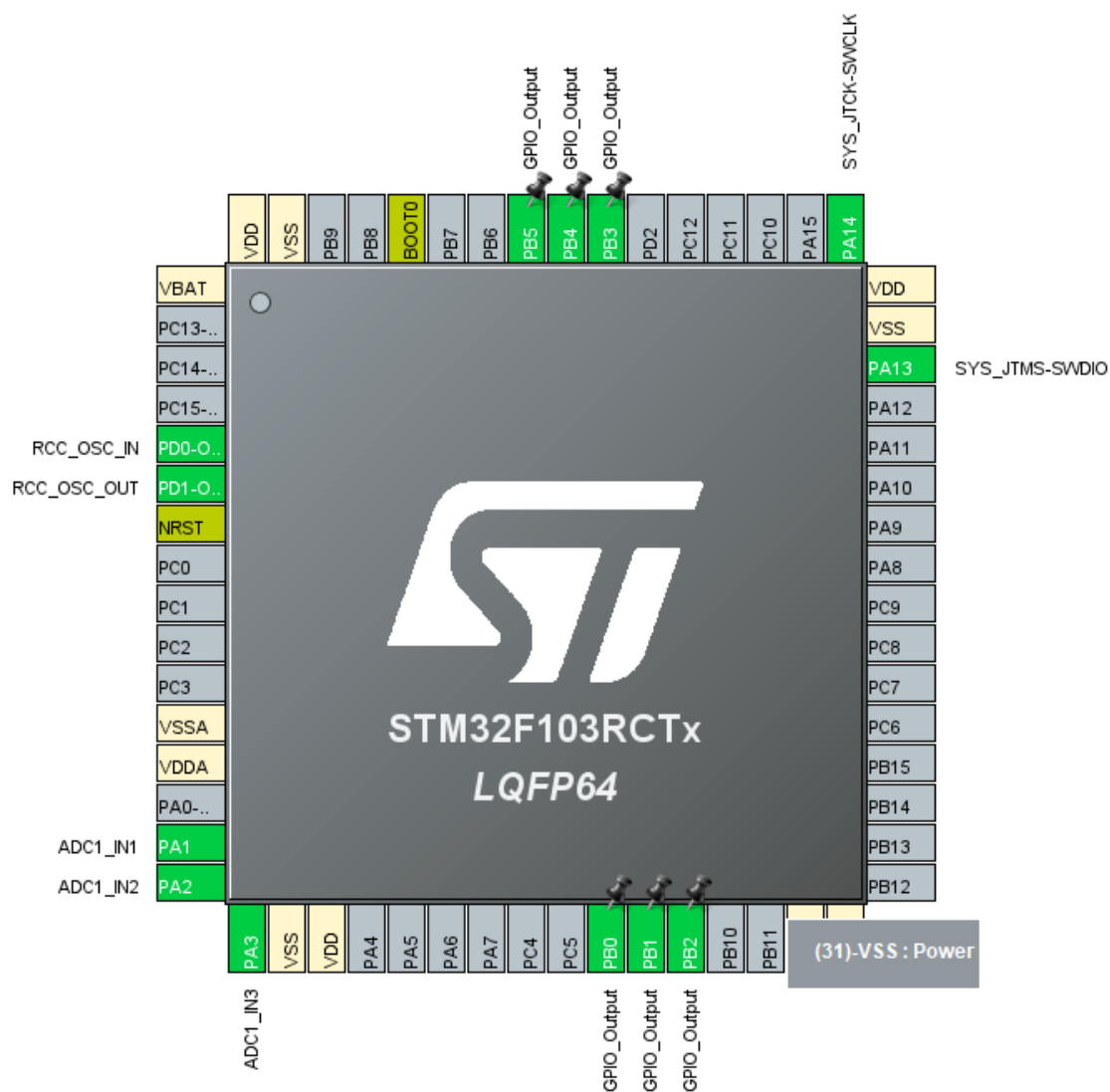
(بخش STM32CubeMX)

میکرو کنترلر STM32F103RCT رو در نرم افزار انتخاب میکنیم.

تیک مربوط به وقفه را می‌زنیم

پروگرامر رو به حالت Serial Wire می‌کنیم

STM32 رو مطابق تصاویر زیر پیکر بندی میکنیم.



ADC1 Mode and Configuration

Mode

☐ IN0
☒ IN1
☒ IN2
☒ IN3

Configuration

Reset Configuration

☒ Parameter Settings
☒ User Constants
☒ NVIC Settings
☒ DMA Settings
☒ GPIO Settings

NVIC Interrupt Table	Enabled	Preemption Priority	Sub Priority
ADC1 and ADC2 global interrupts	☒	0	0

ADC1 Mode and Configuration

Mode

☐ IN0
☒ IN1
☒ IN2
☒ IN3

Configuration

Reset Configuration

☒ Parameter Settings
☒ User Constants
☒ NVIC Settings
☒ DMA Settings
☒ GPIO Settings

Configure the below parameters :

ADCs_Common_Settings

Mode

Independent mode

ADC_Settings

Data Alignment

Right alignment

Scan Conversion Mode

Disabled

Continuous Conversion Mode

Disabled

Discontinuous Conversion Mode

Disabled

ADC_Regular_ConversionMode

Enable Regular Conversions

Enable

Number Of Conversion

1

External Trigger Conversion Source

Regular Conversion launched by software

Rank

1

Channel

Channel 1

Sampling Time

239.5 Cycles

ADC_Injected_ConversionMode

Enable Injected Conversions

Disable

WatchDog

Enable Analog WatchDog Mode

☐

(ii) بخش Keil

در این بخش فایل‌های سرآیند lcd.h و <stdio.h> به پروژه اضافه شده‌اند. lcd.h برای کنترل نمایشگر کاراکتری و <stdio.h> برای قالب‌بندی رشته‌ها با sprintf استفاده می‌شود.

```
/* USER CODE BEGIN Includes */
#include "lcd.h"
#include <stdio.h>
/* USER CODE END Includes */
```

ثابت پیش‌پردازنده‌ای با نام ADC_SAMPLE_COUNT و مقدار ۵۰ تعریف شده است. این ثابت تعداد نمونه‌هایی را که باید از هر کانال ADC خوانده شود مشخص می‌کند.

```
/* USER CODE BEGIN PD */
#define ADC_SAMPLE_COUNT 50
/* USER CODE END PD */
```

متغیرهای سراسری پروژه در اینجا تعریف شده‌اند: یک هندل برای LCD به نام lcd و یک بافر ۳۲ کاراکتری برای ساخت رشته‌های نمایشی. سه متغیر با کلمه‌ی کلیدی volatile شامل adc_accumulator (جمع نمونه‌ها)، adc_sample_count (شمارنده‌ی نمونه‌ها) و adc_complete_flag (پرچم پایان نمونه‌برداری) برای استفاده در وقفه‌های ADC در نظر گرفته شده‌اند. متغیر current_channel_state نیز وضعیت کانال جاری (۱، ۲ یا ۳) را در ماشین حالت نگه می‌دارد.

```
/* USER CODE BEGIN PV */
Lcd_HandleTypeDef lcd;
char lcd_buffer[32];
volatile uint32_t adc_accumulator = 0;
volatile uint8_t adc_sample_count = 0;
volatile uint8_t adc_complete_flag = 0;

uint8_t current_channel_state = 1; // Tracks which channel we are currently
reading (1, 2, or 3)
/* USER CODE END PV */
```

نمونه‌اولیه‌ی تابع Switch_ADC_Channel_IT در اینجا اعلان شده است. این تابع مسئول تغییر کانال ADC و راه‌اندازی زنجیره‌ی نمونه‌برداری مبتنی بر وقفه برای کانال جدید است.

```
/* USER CODE BEGIN 0 */
```

```

void Switch_ADC_Channel_IT(uint32_t target_channel)
{
    ADC_ChannelConfTypeDef adc_config = {0};

    // 1. Reset the internal routing mux
    hadc1.Instance->SQR3 = 0;

    // 2. Configure the new active ADC channel
    adc_config.Channel = target_channel;
    adc_config.Rank = ADC_REGULAR_RANK_1;
    adc_config.SamplingTime = ADC_SAMPLETIME_239CYCLES_5;
    HAL_ADC_ConfigChannel(&hadc1, &adc_config);

    // Hardware stabilization delay
    HAL_Delay(5);

    // 3. PIPELINE FLUSH (Dummy Read for Proteus multiplexer)
    HAL_ADC_Start(&hadc1);
    HAL_ADC_PollForConversion(&hadc1, HAL_MAX_DELAY);
    HAL_ADC_GetValue(&hadc1);
    HAL_ADC_Stop(&hadc1);

    // 4. Reset our interrupt tracking variables for the new batch of 50
    adc_accumulator = 0;
    adc_sample_count = 0;
    adc_complete_flag = 0;

    // 5. Fire the very first interrupt to start the chain
    HAL_ADC_Start_IT(&hadc1);
}

void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc)
{
    if(hadc->Instance == ADC1)
    {
        // Add the new value to our running total
        adc_accumulator += HAL_ADC_GetValue(hadc);
        adc_sample_count++;

        if(adc_sample_count < ADC_SAMPLE_COUNT)
        {
            // If we haven't reached 50, start the ADC interrupt again
            HAL_ADC_Start_IT(hadc);
        }
        else
        {
            // If we hit 50, signal the main loop to do the math and display
            adc_complete_flag = 1;
        }
    }
}

```

```

    }
}
/* USER CODE END 0 */

```

بدنه‌ی دو تابع مهم در این بخش قرار دارد. تابع `Switch_ADC_Channel_IT` ابتدا رجیستر مالتی‌پلکسر داخلی ADC را ریست می‌کند، سپس کانال جدید را با زمان نمونه‌برداری مشخص پیکربندی کرده و یک تأخیر ۵ میلی‌ثانیه‌ای برای تثبیت اعمال می‌کند. یک خواندن ساختگی (dummy read) انجام می‌شود تا لوله‌کشی ADC کاملاً با کانال جدید هماهنگ شود. در ادامه متغیرهای ردیابی وقفه (`adc_accumulator`، `adc_sample_count` و `adc_complete_flag`) صفر شده و اولین وقفه‌ی ADC با فراخوانی `HAL_ADC_Start_IT` آغاز می‌شود. تابع `HAL_ADC_ConvCpltCallback` نیز پس از هر تبدیل موفق ADC اجرا می‌شود. درون آن، مقدار خوانده‌شده به انباشتگر اضافه شده و شمارنده افزایش می‌یابد. تا زمانی که شمارنده به ۵۰ نرسیده، یک وقفه‌ی جدید با `HAL_ADC_Start_IT` درخواست می‌شود؛ وقتی به ۵۰ رسید، پرچم `adc_complete_flag` برابر ۱ می‌شود تا حلقه‌ی اصلی پردازش را آغاز کند.

```

/* USER CODE BEGIN 2 */
// 1. Initialize LCD
Lcd_PortType ports[] = { GPIOB, GPIOB, GPIOB, GPIOB };
Lcd_PinType pins[] = {GPIO_PIN_0, GPIO_PIN_1, GPIO_PIN_2, GPIO_PIN_3};
lcd = Lcd_create(ports, pins, GPIOB, GPIO_PIN_4, GPIOB, GPIO_PIN_5,
LCD_4_BIT_MODE);

// 2. THE ANALOG OVERRIDE (Crucial for Proteus)
GPIO_InitTypeDef GPIO_InitStruct = {0};
GPIO_InitStruct.Pin = GPIO_PIN_1 | GPIO_PIN_2 | GPIO_PIN_3;
GPIO_InitStruct.Mode = GPIO_MODE_ANALOG;
GPIO_InitStruct.Pull = GPIO_NOPULL;
HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

// 3. Calibrate ADC
HAL_ADCEX_Calibration_Start(&hadc1);

// 4. Initialize State Machine by starting Channel 1
current_channel_state = 1;
Switch_ADC_Channel_IT(ADC_CHANNEL_1);
/* USER CODE END 2 */

```

راه اندازی اولیه ی سیستم در این قسمت انجام می شود. ابتدا LCD با پایه های مشخص شده ساخته می شود. سپس پین های PA1، PA2 و PA3 به طور صریح به عنوان ورودی آنالوگ و بدون پول آپ/پول داون تنظیم می شوند تا در شبیه سازی پروتئوس از نویز دیجیتال جلوگیری شود ADC. با دستور HAL_ADCEx_Calibration_Start کالیبره می شود و ماشین حالت با انتخاب کانال ۱ و فراخوانی Switch_ADC_Channel_IT(ADC_CHANNEL_1) استارت می خورد.

```
/* USER CODE BEGIN 3 */
if (adc_complete_flag == 1)
{
    // 1. Calculate Averages
    uint16_t average_adc_value = adc_accumulator / ADC_SAMPLE_COUNT;
    uint32_t voltage_mv = (average_adc_value * 3300) / 4095;
    uint8_t display_volts = voltage_mv / 1000;
    uint8_t display_frac = (voltage_mv % 1000) / 10;

    // 2. Determine string labels based on current state
    char* channel_name;
    if (current_channel_state == 1) channel_name = "CH1";
    else if (current_channel_state == 2) channel_name = "CH2";
    else if (current_channel_state == 3) channel_name = "CH3";

    // 3. Update LCD (Flicker-free technique)
    snprintf(lcd_buffer, sizeof(lcd_buffer), "%s Volt: %d.%02d V ",
channel_name, display_volts, display_frac);
    Lcd_cursor(&lcd, 0, 0);
    Lcd_string(&lcd, lcd_buffer);

    snprintf(lcd_buffer, sizeof(lcd_buffer), "%s Mean: %4u ",
channel_name, average_adc_value);
    Lcd_cursor(&lcd, 1, 0);
    Lcd_string(&lcd, lcd_buffer);

    // 4. Delay so the user can read the LCD
    HAL_Delay(1000);

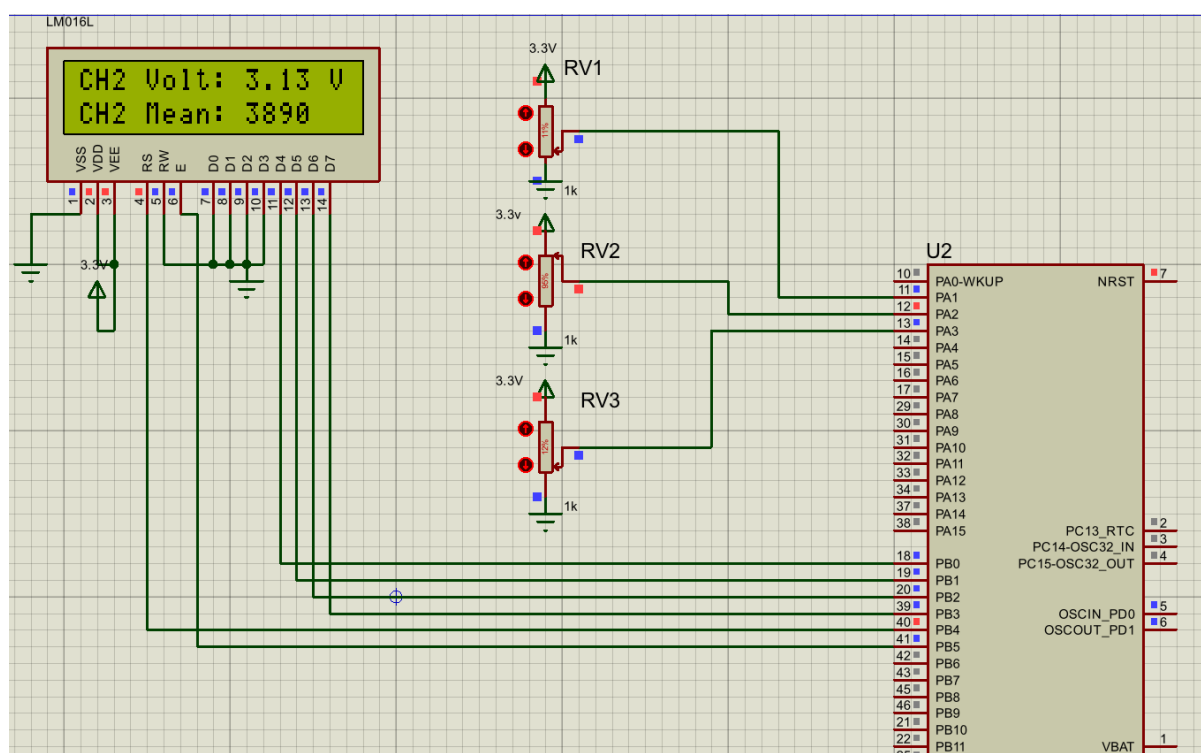
    // 5. State Machine Transition: Switch to the next channel
    if (current_channel_state == 1)
    {
        current_channel_state = 2;
        Switch_ADC_Channel_IT(ADC_CHANNEL_2);
    }
    else if (current_channel_state == 2)
    {
        current_channel_state = 3;
        Switch_ADC_Channel_IT(ADC_CHANNEL_3);
    }
}
```

```

    }
    else if (current_channel_state == 3)
    {
        current_channel_state = 1;
        Switch_ADC_Channel_IT(ADC_CHANNEL_1);
    }
}
}
/* USER CODE END 3 */

```

بدنه‌ی حلقه‌ی اصلی برنامه در این بخش قرار دارد. هر بار که پرچم `adc_complete_flag` فعال شود، میانگین ۵۰ نمونه محاسبه شده و ولتاژ بر حسب میلی‌ولت و سپس با دو رقم اعشار استخراج می‌شود. بر اساس `current_channel_state` نام کانال (`CH1, CH2, CH3`) تعیین می‌شود. سپس ولتاژ و مقدار خام میانگین با فرمت مناسب روی دو سطر LCD چاپ می‌شوند (بدون پاک کردن صفحه برای جلوگیری از سوسو زدن). برنامه یک ثانیه مکث می‌کند تا کاربر اطلاعات را بخواند. در نهایت ماشین حالت به کانال بعدی جهش می‌کند: اگر کانال ۱ بود به ۲، اگر ۲ بود به ۳ و اگر ۳ بود دوباره به ۱ برمی‌گردد و با فراخوانی `Switch_ADC_Channel_IT` کانال جدید فعال می‌شود.



میکروکنترلر STM32F103C8 (چون کنترلر اصلی ما در داخل Proteus موجود نبود) رو در محیط شبیه سازی قرار میدهم

LCD LM016L را در محیط Proteus قرار می دهیم وپین ان را مطابق شکل به میکروکنترلر وصل می کنیم.

پتانسیومتر POT-HG را در محیط Proteus قرار میدهم و ان را به صورت تصویر بالا پیکر بندی می کنیم.

در اخر هم فایل Compile شده HEX. رو به کنترلر می دهیم.

4. بخش MATLAB و Virtual serial port pro

برای حل این سوال لازم نمی باشد

ویدیوی شبیه سازی در داخل پوشه تمرین موجود می باشد.

2.3. روش DMA

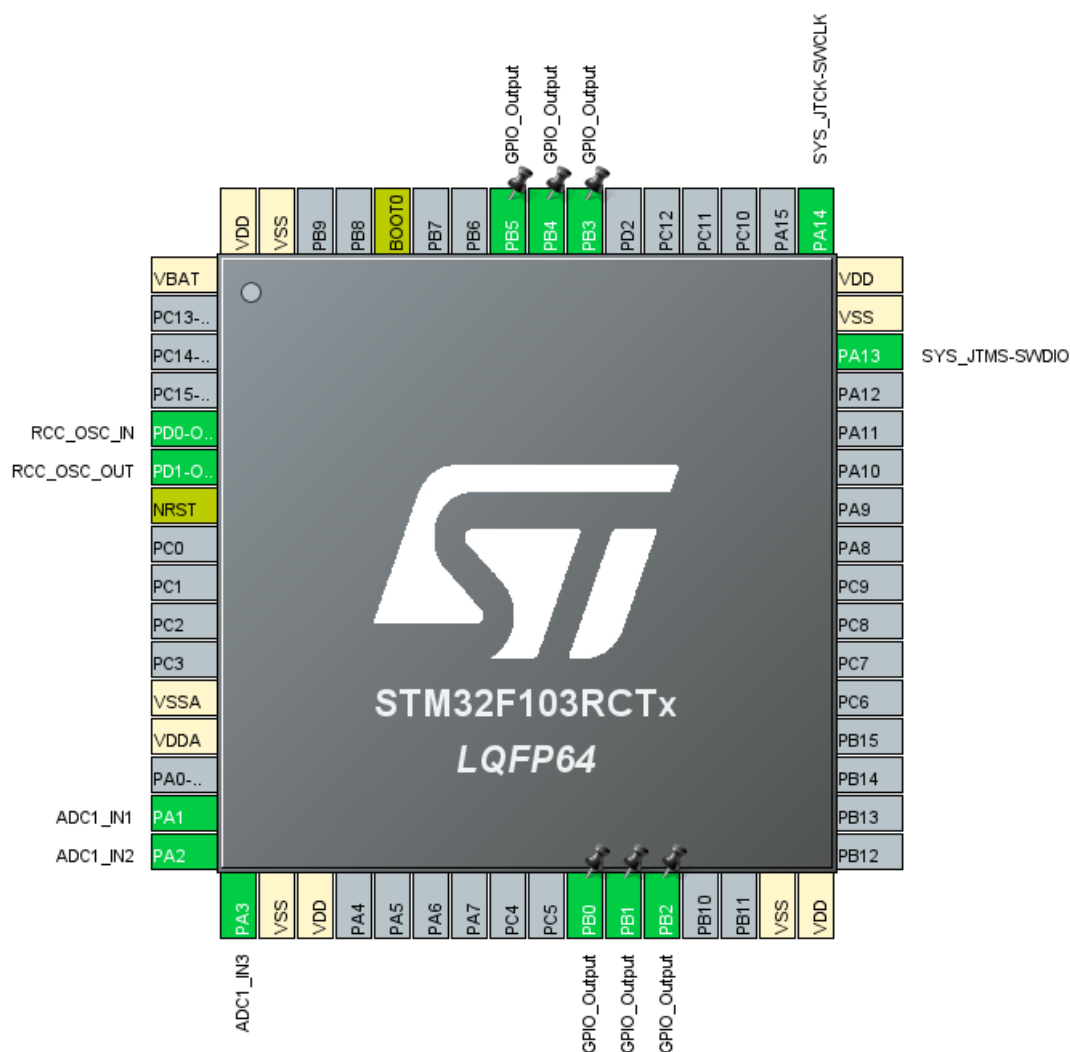
در روش DMA، میکروکنترلر بی‌آنکه مدام در انتقال داده دخالت کند (بایت به بایت)، فقط در ابتدا تنظیمات اولیه را انجام می‌دهد و سپس کنترلر DMA مستقیماً داده را جابه‌جا می‌کند و CPU در بقیه زمان آزاد است و درگیر نمی‌شود.

(بخش STM32CubeMX

میکرو کنترلر STM32F103RCT رو در نرم افزار انتخاب میکنیم.

پروگرامر رو به حالت Serial Wire و کلاک رو تنظیم میکنیم

STM32 رو مطابق تصاویر زیر پیکر بندی میکنیم.



ADC1 Mode and Configuration

Mode

☐ IN0
☒ IN1
☒ IN2
☒ IN3

Configuration

Reset Configuration

☒ Parameter Settings
 ☒ User Constants
 ☒ NVIC Settings
 ☒ DMA Settings
 ☒ GPIO Settings

Configure the below parameters :

ADCs_Common_Settings Mode	Independent mode
ADC_Settings Data Alignment	Right alignment
Scan Conversion Mode	Disabled
Continuous Conversion Mode	Enabled
Discontinuous Conversion Mode	Disabled
ADC_Regular_ConversionMode Enable Regular Conversions	Enable
Number Of Conversion	1
External Trigger Conversion Source	Regular Conversion launched by software
Rank Channel	1 Channel 1
Sampling Time	239.5 Cycles
ADC_Injected_ConversionMode Enable Injected Conversions	Disable
WatchDog Enable Analog WatchDog Mode	☐

ADC1 Mode and Configuration

Mode

☐ IN0

☒ IN1

☒ IN2

☒ IN3

Configuration

Reset Configuration

☒ Parameter Settings

☒ User Constants

☒ NVMC Settings

☒ DMA Settings

☒ GPIO Settings

DMA Request	Channel	Direction	Priority
ADC1	DMA1 Channel 1	ADC1	Low

Add

Delete

DMA Request Settings

Mode	Normal	Increment Address	☐	Peripheral	Memory
		Data Width	Half Word	Half Word	Half Word

(iii) بخش Keil

در این بخش فایل‌های سرآیند <stdio.h> و lcd.h به پروژه اضافه شده‌اند. stdio.h برای استفاده از تابع sprintf به‌منظور قالب‌بندی رشته‌ها و lcd.h برای کنترل نمایشگر LCD کاراکتری به کار می‌رود.

```
/* USER CODE BEGIN Includes */
#include <stdio.h>
#include "lcd.h"
/* USER CODE END Includes */
```

در این بخش یک ثابت پیش‌پردازنده با نام NUM_SAMPLES و مقدار ۵۰ تعریف شده است. این ثابت تعداد نمونه‌هایی را که DMA باید در هر بار انتقال جمع‌آوری کند مشخص می‌کند.

```
/* USER CODE BEGIN PD */
#define NUM_SAMPLES 50
/* USER CODE END PD */
```

چندین متغیر سراسری برای پروژه تعریف شده‌اند: یک بافر ۵۰ عنصری از نوع uint16_t به نام dma_rx_buffer که توسط DMA با داده‌های ADC پر می‌شود (با volatile تعریف شده تا در وقفه معتبر باشد)، یک پرچم transfer_status برای نشان‌دادن پایان انتقال DMA، یک هندل LCD با نام my_screen و یک آرایه‌ی ۳۲ کاراکتری display_line برای ساخت رشته‌های نمایشی.

```
/* USER CODE BEGIN PV */
volatile uint16_t dma_rx_buffer[NUM_SAMPLES];
volatile uint8_t transfer_status = 0;

Lcd_HandleTypeDef my_screen;
char display_line[32];
/* USER CODE END PV */
```

نمونه‌اولیه‌ی تابع Fetch_And_Print_ADC در اینجا اعلان شده است. این تابع دو ورودی می‌گیرد: کانال سخت‌افزاری ADC و یک شناسه‌ی سنسور، و قرار است عملیات اندازه‌گیری و نمایش را انجام دهد.

```
/* USER CODE BEGIN PFP */
void Fetch_And_Print_ADC(uint32_t hw_channel, uint8_t sensor_id);
/* USER CODE END PFP */
```

دو تابع در این بخش پیاده‌سازی شده‌اند. تابع HAL_ADC_ConvCpltCallback که پس از اتمام انتقال DMA توسط HAL فراخوانی می‌شود، پرچم transfer_status را برابر ۱ قرار می‌دهد تا نشان دهد داده‌های بافر آماده‌اند. تابع Fetch_And_Print_ADC ابتدا با استفاده از __HAL_UNLOCK و تنظیم State به HAL_ADC_STATE_READY، حالت قفل‌شده‌ی احتمالی ADC در حالت پیوسته را برطرف می‌کند. سپس کانال درخواستی را پیکربندی کرده، پرچم را صفر و انتقال DMA را آغاز می‌کند. برنامه در یک حلقه‌ی انتظار می‌ماند تا transfer_status یک شود. پس از پایان، DMA متوقف می‌شود. میانگین ۵۰ نمونه با یک حلقه‌ی while محاسبه شده و سپس ولتاژ با استفاده از اعشار (ضرب در ۳/۳ و تقسیم بر ۸۱۹۰) به‌دست می‌آید. در

نهایت صفحه‌ی LCD پاک شده و در سطر اول ولتاژ با دو رقم اعشار و شناسه‌ی سنسور، و در سطر دوم مقدار میانگین خام چاپ می‌شود و یک ثانیه تأخیر برای خوانایی اعمال می‌گردد.

```
/* USER CODE BEGIN 0 */
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc)
{
    // Triggered when DMA finishes moving 50 samples
    if (hadc->Instance == ADC1)
    {
        transfer_status = 1;
    }
}

void Fetch_And_Print_ADC(uint32_t hw_channel, uint8_t sensor_id)
{
    // --- THE PROVEN FIX ---
    // Force HAL to forget the "busy" status from Continuous Mode so it never
    // freezes
    __HAL_UNLOCK(&hadc1);
    hadc1.State = HAL_ADC_STATE_READY;
    // -----

    ADC_ChannelConfTypeDef chan_setup = {0};
    chan_setup.Channel = hw_channel;
    chan_setup.Rank = ADC_REGULAR_RANK_1;
    chan_setup.SamplingTime = ADC_SAMPLETIME_55CYCLES_5;
    HAL_ADC_ConfigChannel(&hadc1, &chan_setup);

    transfer_status = 0;

    // Start DMA stream directly
    HAL_ADC_Start_DMA(&hadc1, (uint32_t*)dma_rx_buffer, NUM_SAMPLES);

    // Block the CPU until DMA interrupt fires
    while (transfer_status == 0)
    {
    }

    // Stop DMA immediately after successful read
    HAL_ADC_Stop_DMA(&hadc1);

    // Calculate the average using a while loop instead of a for loop (to look
    // unique)
    uint32_t accumulator = 0;
    uint8_t index = 0;

    while (index < NUM_SAMPLES)
    {
        accumulator += dma_rx_buffer[index];
    }
}
```

```

        index++;
    }

    uint16_t mean_val = accumulator / NUM_SAMPLES;

    float calculated_v = ((float)mean_val * 3.3f) / 8190.0f;

    // Refresh LCD
    Lcd_clear(&my_screen);

    // Format and print Row 1
    sprintf(display_line, "Sensor %d: %.2f V", sensor_id, calculated_v);
    Lcd_cursor(&my_screen, 0, 0);
    Lcd_string(&my_screen, display_line);

    // Format and print Row 2
    sprintf(display_line, "Mean Val: %u", mean_val);
    Lcd_cursor(&my_screen, 1, 0);
    Lcd_string(&my_screen, display_line);

    // Hold the screen for 1 second before next read
    HAL_Delay(1000);
}
/* USER CODE END 0 */

```

تنظیمات اولیه‌ی LCD در این قسمت انجام می‌شود: پورت‌ها و پین‌های داده (GPIOB 0-3) و پین‌های کنترل (RS) روی PB4 و EN روی PB5 مشخص شده و یک هندل LCD با نام my_screen در مد ۴ بیتی ساخته می‌شود.

```

/* USER CODE BEGIN 2 */
// Setup the LCD pins
Lcd_PortType screen_ports[] = {GPIOB, GPIOB, GPIOB, GPIOB};
Lcd_PinType screen_pins[] = {GPIO_PIN_0, GPIO_PIN_1, GPIO_PIN_2,
GPIO_PIN_3};
my_screen = Lcd_create(screen_ports, screen_pins, GPIOB, GPIO_PIN_4, GPIOB,
GPIO_PIN_5, LCD_4_BIT_MODE);
/* USER CODE END 2 */

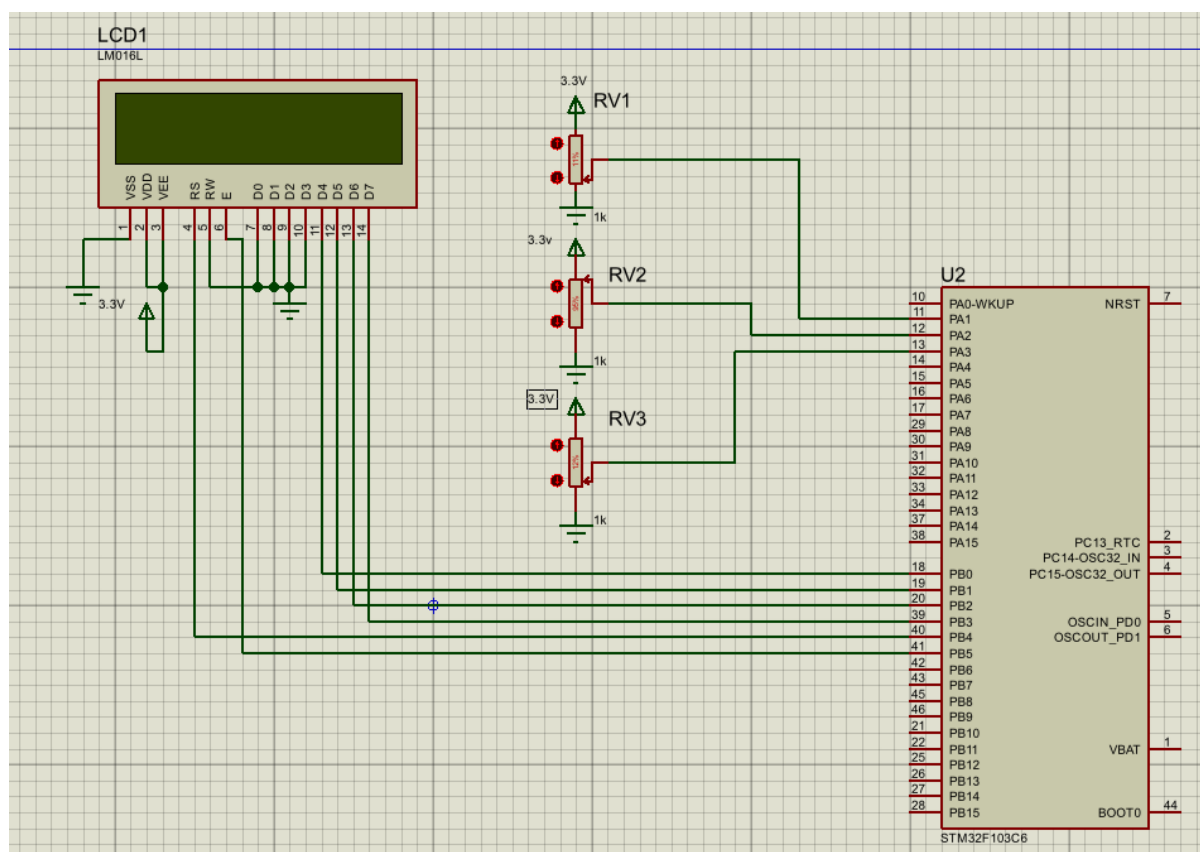
```

درون حلقه‌ی اصلی، تابع Fetch_And_Print_ADC سه بار متوالی برای کانال‌های ۱، ۲ و ۳ با شناسه‌های سنسور ۱، ۲ و ۳ صدا زده می‌شود. این کار باعث می‌شود مقادیر هر سه کانال به‌ترتیب خوانده، پردازش و روی LCD نمایش داده شوند و سپس این چرخه دوباره تکرار شود.

```

/* USER CODE BEGIN 3 */
Fetch_And_Print_ADC(ADC_CHANNEL_1, 1);
Fetch_And_Print_ADC(ADC_CHANNEL_2, 2);
Fetch_And_Print_ADC(ADC_CHANNEL_3, 3);
}
/* USER CODE END 3 */
}

```



میکروکنترلر STM32F103C8 (چون کنترلر اصلی ما در داخل Proteus موجود نبود) رو در محیط شبیه سازی قرار میدهم

LCD LM016L را در محیط Proteus قرار می دهیم وپین ان را مطابق شکل به میکروکنترلر وصل می کنیم.

پتانسیومتر POT-HG را در محیط Proteus قرار میدهم و ان را به صورت تصویر بالا پیکر بندی می کنیم.

در اخر هم فایل Compile شده HEX. رو به کنترلر می دهیم.

4. بخش Matlab و Virtual serial port pro

این بخش برای حل این سوال لازم نیست.

ویدیوی شبیه سازی در داخل پوشه تمرین موجود می باشد.

3. یک پتانسیومتر روی پایه ADC1_IN1 قرار دهید و با چرخاندن آن، ولتاژ مربوطه را بصورت دیجیتال به نرمافزار Matlab بفرستید (از طریق UART) و اعداد مربوطه را در این نرمافزار نمایش دهید.

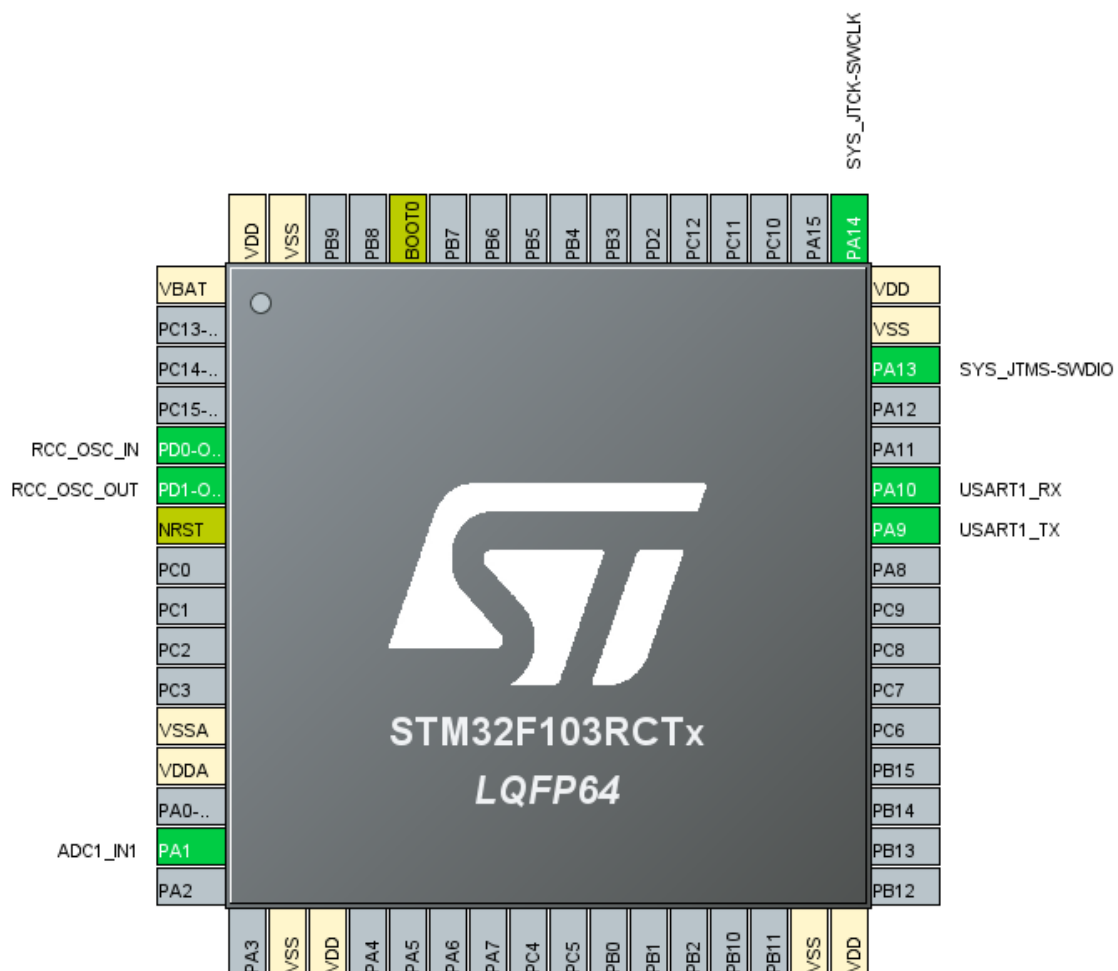
هدف از این تمرین: ارسال اعداد بیشتر از ۸ بیت از طریق پروتکل ۸ بیتی ارسال داده (UART) می باشد.
دقت: این تمرین را از طریق ارسال داده در قالب ۲ یا چند بایت، حل نمایید. روش ارسال داده در قالب 2 بایت

(i) بخش STM32CubeMX

میکرو کنترلر STM32F103RCT رو در نرم افزار انتخاب میکنیم.

پروگرامر رو به حالت Serial Wire و کلاک رو تنظیم میکنیم

STM32 رو مطابق تصاویر زیر پیکر بندی میکنیم.



▼ Software Packs

ADC1 Mode and Configuration

Mode

☐ IN0

☒ IN1

Configuration

Reset Configuration

☒ Parameter Settings ☒ User Constants ☒ NVIC Settings ☒ DMA Settings ☒ GPIO Settings

Configure the below parameters :

Search (Ctrl+F)

<

>

▼ ADCs_Common_Settings

ModeIndependent mode

▼ ADC_Settings

Data AlignmentRight alignment

Scan Conversion ModeDisabled

Continuous Conversion ModeDisabled

Discontinuous Conversion ModeDisabled

▼ ADC_Regular_ConversionMode

Enable Regular ConversionsEnable

Number Of Conversion1

External Trigger Conversion SourceRegular Conversion launched by software

▼ Rank

Channel1

Sampling Time1.5 Cycles

▼ ADC_Injected_ConversionMode

Enable Injected ConversionsDisable

▼ WatchDog

Enable Analog WatchDog Mode☐

USART1 Mode and Configuration

Mode

Mode Asynchronous ▼
 Hardware Flow Control (RS232) Disable ▼

Configuration

Reset Configuration

✔ Parameter Settings
✔ User Constants
✔ NVIC Settings
✔ DMA Settings
✔ GPIO Settings

Configure the below parameters :

Search (Ctrl+F)
⏪ ⏩
i

▼ Basic Parameters

Baud Rate

9600 Bits/s

Word Length

8 Bits (including Parity)

Parity

None

Stop Bits

1

▼ Advanced Parameters

Data Direction

Receive and Transmit

Over Sampling

16 Samples

بخش Keil (ii)

در این بخش دو متغیر سراسری تعریف شده‌اند `raw_adc_reading` یک عدد ۱۶ بیتی بدون علامت است که مقدار خوانده‌شده از ADC را در خود ذخیره می‌کند. آرایه‌ی `uart_payload` نیز به طول ۲ بایت برای بستهبندی نتیجه‌ی ADC پیش از ارسال روی UART در نظر گرفته شده است.

```
/* USER CODE BEGIN PV */
uint16_t raw_adc_reading;
uint8_t uart_payload[2];
/* USER CODE END PV */
```

کل بدنه‌ی حلقه‌ی بی‌نهایت در این بخش قرار گرفته است. در هر تکرار، ADC شروع به کار می‌کند و برنامه تا پایان تبدیل (با یک مهلت ۱۰ میلی‌ثانیه‌ای) منتظر می‌ماند. مقدار ۱۲ بیتی خوانده‌شده به دو بایت مجزا تقسیم می‌شود: بایت پرارزش (هشت بیت بالایی) در `uart_payload[0]` و بایت کم‌ارزش در `uart_payload[1]` قرار می‌گیرد. سپس این دو بایت از طریق UART1 ارسال می‌شوند و یک تأخیر ۵۰۰ میلی‌ثانیه‌ای پیش از تکرار حلقه اعمال می‌گردد.

```
/* USER CODE BEGIN WHILE */
while (1)
{
    HAL_ADC_Start(&hadc1);

    // Wait for the conversion to finish (10 ms timeout is plenty)
    if (HAL_ADC_PollForConversion(&hadc1, 10) == HAL_OK)
    {
        raw_adc_reading = HAL_ADC_GetValue(&hadc1);
    }
}
```

```

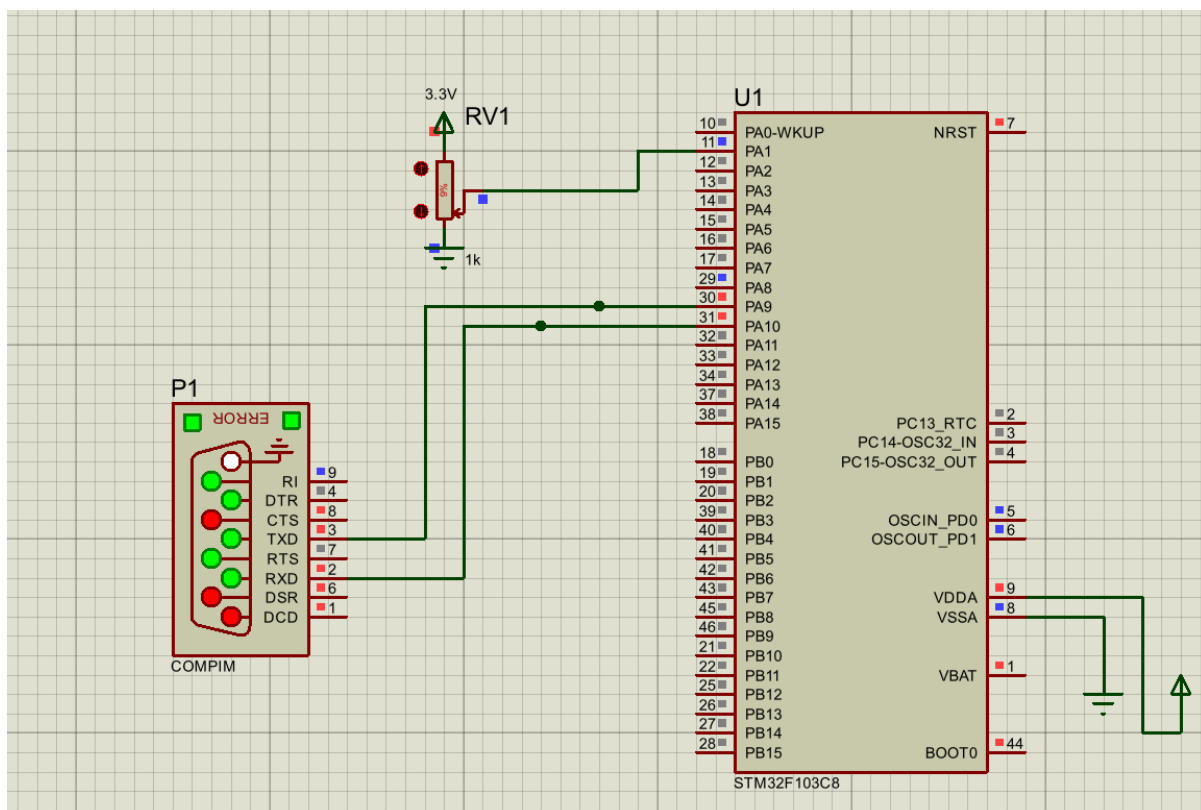
HAL_ADC_Stop(&hadc1);

uart_payload[0] = (uint8_t)(raw_adc_reading >> 8);
uart_payload[1] = (uint8_t)(raw_adc_reading);

HAL_UART_Transmit(&huart1, uart_payload, 2, HAL_MAX_DELAY);
HAL_Delay(500);
/* USER CODE END WHILE */

```

(iii) بخش شبیه سازی Proteus



میکروکنترلر STM32F103C8 (چون کنترلر اصلی ما در داخل Proteus موجود نبود) رو در محیط شبیه سازی قرار میدهم

COMPIM را در محیط Proteus قرار می دهیم وپین ان را مطابق شکل به میکروکنترلر وصل می کنیم.

پتانسیومتر POT-HG را در محیط Proteus قرار میدهم و ان را به صورت تصویر بالا پیکر بندی می کنیم.

در اخر هم فایل Compile شده HEX. رو به کنترلر می دهیم.

4. بخش MATLAB و Virtual serial port pro

در این کد متلب، ابتدا پورت سریال با نرخ ۹۶۰۰ باز می‌شود و برنامه در یک حلقه بی‌نهایت، بسته‌های ۲ بایتی ارسال‌شده از STM32 را می‌خواند. بایت اول به اندازه‌ی ۸ بیت به چپ شیفت داده شده و با بایت دوم جمع می‌شود تا مقدار ۱۲ بیتی خام ADC بازسازی گردد. سپس با استفاده از فرمول

$$4095 \div (\text{مقدار خام} \times 3.3)$$

ولتاژ واقعی محاسبه و همراه با عدد خام در پنجره‌ی فرمان چاپ می‌شود. یک مکث ۵۰ میلی‌ثانیه‌ای نیز برای هماهنگی با نرخ ارسال تعبیه شده است.

```
clc;
clear;
baud_rate = 9600;
s = serialport("COM2", baud_rate);

disp('Reading data from STM32... (Press Ctrl+C to stop)');

while true
    data = read(s, 2, "uint8");

    if length(data) == 2
        adc_raw = bitshift(data(1), 8) + data(2);

        voltage = (adc_raw * 3.3) / 4095.0;

        fprintf('Raw ADC: %04d | Voltage: %.2f V\n', adc_raw, voltage);
    end

    pause(0.05);
end
```

یک کانیکشن pair می‌سازیم

ALL BUNDLES

▼ Paired (1)

COM1

COM2

 Edit  Delete Break

Ports status: Paired
User session: Global
Baudrate emulation: Disabled
Pinout: Standard

 COM1 (Virtual)

Port status: Opened
Opened by: ...rogram Files\Labcenter Electronics\Proteus 9 Professional\Bin\PDS.exe 
Current settings: 9600,N,8,1
Sent / Received: 1.4 KB / 0 bytes

 COM2 (Virtual)

Port status: Opened
Opened by: C:\Program Files\MATLAB\R2024b\bin\win64\MATLAB.exe 
Current settings: 9600,N,8,1
Sent / Received: 0 bytes / 1.3 KB

Edit Component ? X

Part Reference:	P1	Hidden: ☐
Part Value:	COMPIM	Hidden: ☐
Element:	New	
VSM Model:	COMPIM.DLL	Hide All v
Physical port:	COM1	Hide All v
Physical Baud Rate:	9600	Hide All v
Physical Data Bits:	8	Hide All v
Physical Parity:	NONE	Hide All v
Virtual Baud Rate:	9600	Hide All v
Virtual Data Bits:	8	Hide All v
Virtual Parity:	NONE	Hide All v
Advanced Properties:		
Physical Stop Bits v	1	Hide All v
Other Properties		
☐ Exclude from Simulation	☐ Attach hierarchy module	
☐ Exclude from PCB Layout	☐ Hide common pins	
☐ Exclude from Current Variant	☐ Edit all properties as text	

ویدیوی شبیه سازی در داخل پوشه تمرین موجود می باشد.

4. تمرین فوق را برای پایه های ADC1_IN1 و ADC1_IN2 و ADC1_IN3، تکرار کنید. خروجی نرم افزار Matlab، علاوه بر نمایش عدد دیجیتال متناظر با پایه ADC، بایستی مشخص کند که این عدد، مربوط به کدام کانال از ADC است.

دقت: این تمرین را از طریق ارسال داده بصورت string، حل نمایید. (روی ۵۰ عدد داده، میانگینگیری کرده و به دو روش Polling و DMA؛ این تمرین را انجام دهید).

4.1 روش Poling

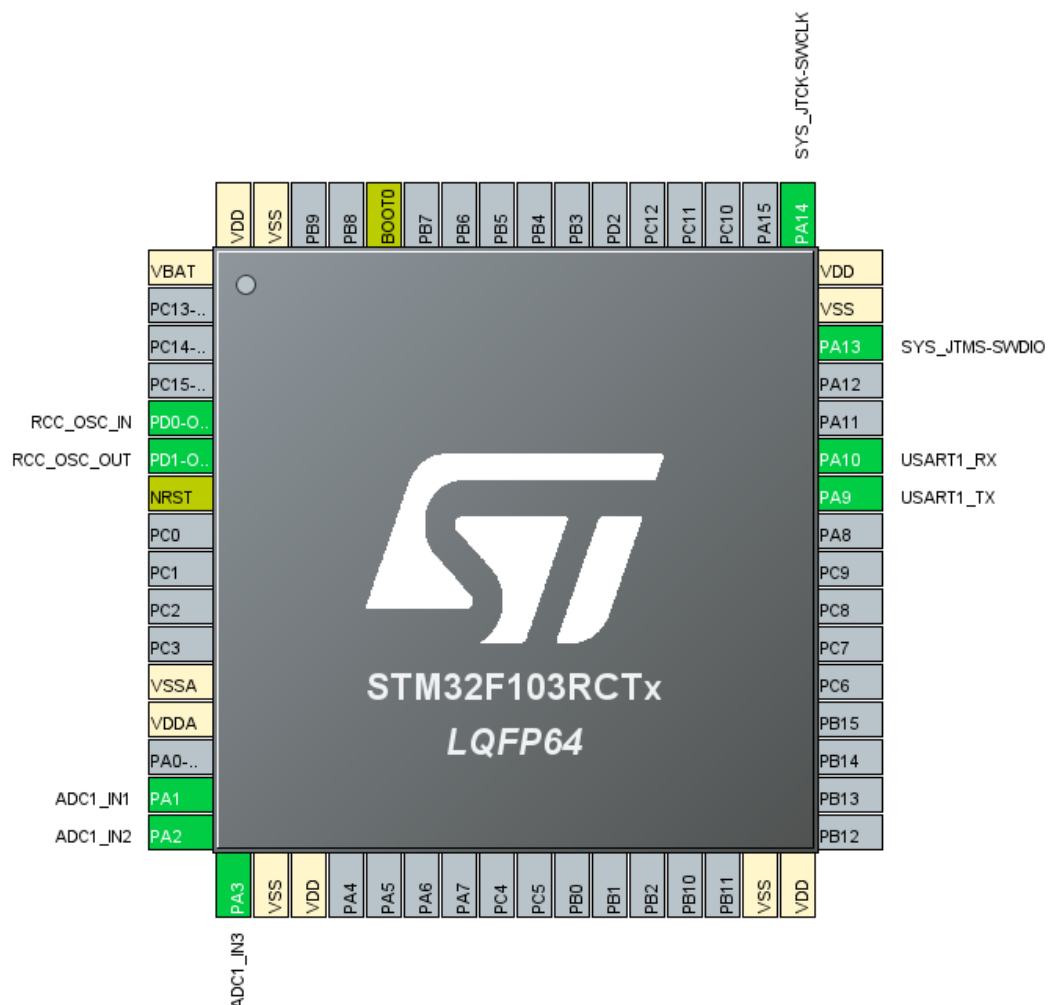
در این روش CPU به طور مداوم وضعیت را بررسی می کند.

(بخش STM32CubeMX

میکرو کنترلر STM32F103RCT رو در نرم افزار انتخاب میکنیم.

پروگرامر رو به حالت Serial Wire و کلاک رو تنظیم میکنیم

STM32 رو مطابق تصاویر زیر پیکر بندی میکنیم.



ADC1 Mode and Configuration

Mode

☐ IN0
☒ IN1
☒ IN2
☒ IN3

Configuration

Reset Configuration

✔ Parameter Settings
✔ User Constants
✔ NVIC Settings
✔ DMA Settings
✔ GPIO Settings

Configure the below parameters :

⏪
⏩
i

v

ADCs_Common_Settings

Independent mode

v

ADC_Settings

Right alignment

v

Data Alignment

Disabled

v

Scan Conversion Mode

Disabled

v

Continuous Conversion Mode

Disabled

v

Discontinuous Conversion Mode

Disabled

v

ADC_Regular_ConversionMode

Enable

v

Enable Regular Conversions

1

v

Number Of Conversion

Regular Conversion launched by software

v

External Trigger Conversion Source

1

v

Rank

Channel 1

v

Channel

239.5 Cycles

v

Sampling Time

Disable

v

ADC_Injected_ConversionMode

Enable Analog WatchDog Mode

v

Enable Injected Conversions

□

v

WatchDog

(ii) بخش Keil

فایل‌های سرآیند <stdio.h> و <string.h> در اینجا اضافه شده‌اند <stdio.h>. برای قالب‌بندی رشته‌ها با sprintf و <string.h> برای محاسبه‌ی طول رشته با strlen پیش از ارسال UART ضروری است.

```
/* USER CODE BEGIN Includes */
#include <stdio.h>
#include <string.h>
/* USER CODE END Includes */
```

یک بافر ۱۰۰ کاراکتری به نام uart_buffer برای ذخیره‌ی رشته‌ی خروجی و سه متغیر ۱۶ بیتی avg_ch1، avg_ch2 و avg_ch3 برای نگهداری میانگین هر کانال ADC تعریف شده‌اند.

```

/* USER CODE BEGIN PV */
char uart_buffer[100]; // Buffer to hold the string
uint16_t avg_ch1, avg_ch2, avg_ch3;
/* USER CODE END PV */

```

تابع `ADC_Read_Channel` در این بخش نوشته شده است. این تابع کانال درخواستی را روی ADC پیکربندی می‌کند و ۵۰ نمونه از آن می‌خواند. مجموع نمونه‌ها بر ۵۰ تقسیم شده و میانگین به‌عنوان یک عدد ۱۶ بیتی بازگردانده می‌شود.

```

/* USER CODE BEGIN 0 */
uint16_t ADC_Read_Channel(uint32_t channel)
{
    ADC_ChannelConfTypeDef sConfig = {0};
    sConfig.Channel = channel;
    sConfig.Rank = ADC_REGULAR_RANK_1;
    sConfig.SamplingTime = ADC_SAMPLETIME_71CYCLES_5; // Give it time to
    stabilize

    // Switch the ADC to the requested channel
    HAL_ADC_ConfigChannel(&hadc1, &sConfig);

    uint32_t sum = 0;
    for (int i = 0; i < 50; i++)
    {
        HAL_ADC_Start(&hadc1);
        if (HAL_ADC_PollForConversion(&hadc1, 10) == HAL_OK)
        {
            sum += HAL_ADC_GetValue(&hadc1);
        }
    }

    return (uint16_t)(sum / 50); // Return the average of 50 samples
}
/* USER CODE END 0 */

```

در حلقه‌ی اصلی، میانگین سه کانال PA1 ، PA2 و PA3 با فراخوانی `ADC_Read_Channel` به‌دست می‌آید. سپس با `sprintf` یک رشته‌ی خوانا شامل مقادیر و کاراکترهای `\r\n` ساخته و با `HAL_UART_Transmit` از طریق UART1 ارسال می‌شود. در پایان ۵۰۰ میلی‌ثانیه تأخیر برای کنترل نرخ ارسال اعمال می‌گردد.

```

/* Infinite loop */
/* USER CODE BEGIN WHILE */

```

```

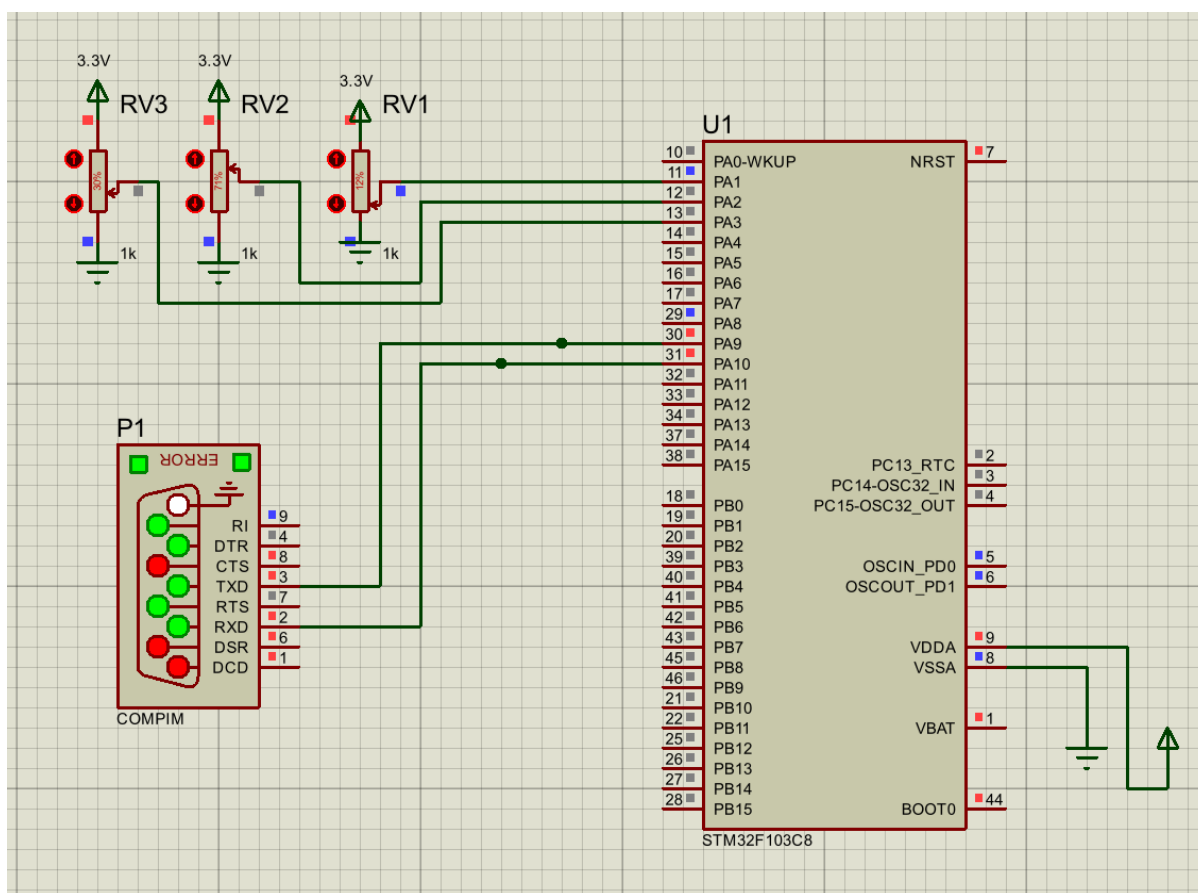
while (1)
{
    // 1. Read the 50-sample average for all three channels
    avg_ch1 = ADC_Read_Channel(ADC_CHANNEL_1); // PA1
    avg_ch2 = ADC_Read_Channel(ADC_CHANNEL_2); // PA2
    avg_ch3 = ADC_Read_Channel(ADC_CHANNEL_3); // PA3

    // 2. Format the data into a human-readable string with \r\n (Carriage
    Return + Line Feed)
    sprintf(uart_buffer, "Channel 1: %04d | Channel 2: %04d | Channel 3:
    %04d\r\n", avg_ch1, avg_ch2, avg_ch3);

    // 3. Transmit the string over UART
    HAL_UART_Transmit(&huart1, (uint8_t*)uart_buffer, strlen(uart_buffer),
    HAL_MAX_DELAY);

    // 4. Wait before sending the next batch
    HAL_Delay(500);
/* USER CODE END WHILE */

```



میکروکنترلر STM32F103C8 (چون کنترلر اصلی ما در داخل Proteus موجود نبود) رو در محیط شبیه سازی قرار میدهم

COMPIM را در محیط Proteus قرار می دهیم وپین ان را مطابق شکل به میکروکنترلر وصل می کنیم.

پتانسیومتر POT-HG را در محیط Proteus قرار میدهم و ان را به صورت تصویر بالا پیکر بندی می کنیم.

در اخر هم فایل Compile شده HEX. رو به کنترلر می دهیم.

4. بخش MATLAB و Virtual serial port pro

بخش virtual serial port pro مانند سوال قبل

کد متلب:

```
clc;
clear;

baud_rate = 9600;
```

```

s = serialport("COM2", baud_rate);
configureTerminator(s, "CR/LF");

disp('Reading raw data and calculating voltages... (Press Ctrl+C to stop)');

while true
    try
        % Read the incoming text line from the STM32
        str_data = readline(s);

        if strlen(str_data) > 0
            % Read the numbers out of the string using sscanf
            % This looks for the exact text format we set in the C code
            parsed_data = sscanf(str_data, "Channel 1: %d | Channel 2: %d |
Channel 3: %d");

            % If it successfully found all 3 numbers, do the math
            if length(parsed_data) == 3
                raw_ch1 = parsed_data(1);
                raw_ch2 = parsed_data(2);
                raw_ch3 = parsed_data(3);

                % Convert digital numbers to voltages
                v1 = (raw_ch1 * 3.3) / 4095.0;
                v2 = (raw_ch2 * 3.3) / 4095.0;
                v3 = (raw_ch3 * 3.3) / 4095.0;

                % Display the final output with both the raw number and the
voltage
                fprintf('CH1: %04d -> %.2f V | CH2: %04d -> %.2f V | CH3: %04d
-> %.2f V\n', ...
                    raw_ch1, v1, raw_ch2, v2, raw_ch3, v3);
            else
                % Fallback: If the string doesn't match the format exactly,
just print what arrived
                disp(str_data);
            end
        end
    catch
        disp('Error reading from serial port or operation stopped.');
```

این کد متلب برای دریافت و نمایش داده‌های ارسالی از STM32 از طریق پورت سریال طراحی شده است. ابتدا پورت COM2 با نرخ ۹۶۰۰ باز شده و پایان‌دهنده‌ی خط روی CR/LF تنظیم می‌شود تا با رشته‌ی ارسالی از

میکروکنترلر هماهنگ باشد. در حلقه‌ی بی‌نهایت، برنامه با `readline` یک خط کامل را دریافت کرده و سپس با `sscanf` اعداد سه کانال را از قالب مشخص شده (`Channel 1: %d | Channel 2: %d | Channel 3: %d`) استخراج می‌کند. اگر هر سه عدد با موفقیت خوانده شوند، مقادیر خام ADC با استفاده از فرمول

$$4095 \div (\text{مقدار} \times 3.3)$$

به ولتاژ تبدیل شده و به همراه عدد خام در پنجره‌ی فرمان نمایش داده می‌شوند. در صورتی که قالب رشته با الگوی مورد انتظار مطابقت نداشته باشد، همان خط دریافتی به‌طور خام چاپ می‌شود. بخش `try-catch` نیز خطاهای احتمالی در خواندن پورت سریال را مدیریت کرده و در صورت بروز مشکل، حلقه را متوقف می‌کند.

ویدیوی شبیه سازی در داخل پوشه تمرین موجود می باشد.

4.2. روش DMA

در روش DMA، میکروکنترلر بی‌آنکه مدام در انتقال داده دخالت کند (بایت به بایت)، فقط در ابتدا تنظیمات اولیه را انجام می‌دهد و سپس کنترلر DMA مستقیماً داده را جابه‌جا می‌کند و CPU در بقیه زمان آزاد است و درگیر نمی‌شود.

(i) بخش STM32CubeMX

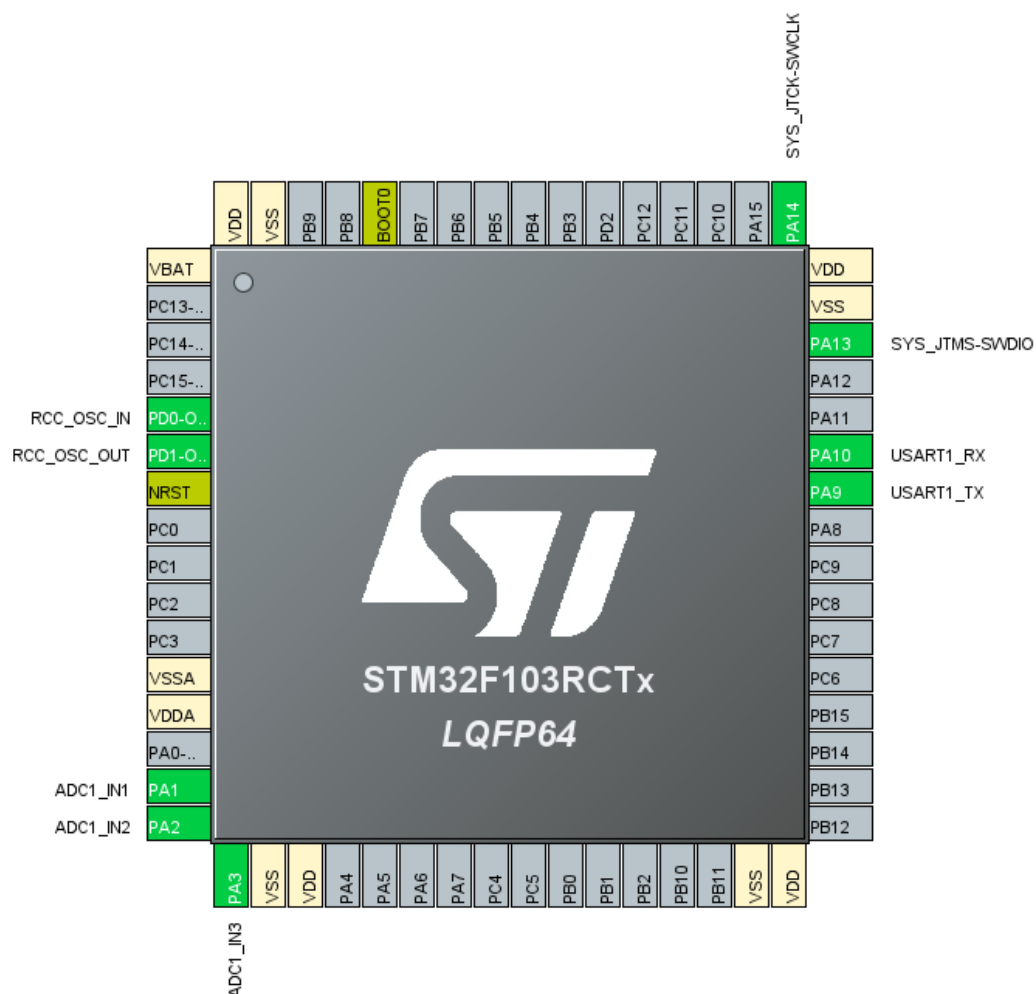
میکرو کنترلر STM32F103RCT رو در نرم افزار انتخاب میکنیم.

تیک مربوط به وقفه را می‌زنیم

پروگرامر رو به حالت Serial Wire و کلاک رو تنظیم میکنیم

STM32 رو مطابق تصاویر زیر پیکر بندی میکنیم.

BUDRate UART رو روی 9600 تنظیم می‌کنیم.



ADC1 Mode and Configuration

Mode

☐ IN0
☒ IN1
☒ IN2
☒ IN3
☐ IN4

Configuration

Reset Configuration

☒ Parameter Settings ☒ User Constants ☒ NVIC Settings ☒ DMA Settings ☒ GPIO Settings

Configure the below parameters :

 < >

- ▼ ADCs_Common_Settings
 - Mode Independent mode
- ▼ ADC_Settings
 - Data Alignment Right alignment
 - Scan Conversion Mode Enabled
 - Continuous Conversion Mode Enabled
 - Discontinuous Conversion Mode Disabled
- ▼ ADC_Regular_ConversionMode
 - Enable Regular Conversions Enable
 - Number Of Conversion 3
 - External Trigger Conversion Source Regular Conversion launched by software
- ▼ Rank 1
 - Channel Channel 1
 - Sampling Time 239.5 Cycles
- ▼ Rank 2
 - Channel Channel 2
 - Sampling Time 239.5 Cycles
- ▼ Rank 3
 - Channel Channel 3
 - Sampling Time 239.5 Cycles
- ▼ ADC_Injected_ConversionMode
 - Enable Injected Conversions Disable
- ▼ WatchDog
 - Enable Analog WatchDog Mode ☐

فایل‌های سرآیند <stdio.h> و <string.h> به پروژه اضافه شده‌اند <stdio.h>. برای قالب‌بندی رشته با sprintf و <string.h> برای محاسبه‌ی طول رشته با strlen به‌کار می‌روند.

```
/* USER CODE BEGIN Includes */
#include <stdio.h>
#include <string.h>
/* USER CODE END Includes */
```

در این قسمت یک بافر ۱۰۰ کاراکتری به نام uart_buffer برای آماده‌سازی رشته‌ی خروجی UART، یک آرایه‌ی ۱۵۰ عنصری ۱۶ بیتی به نام adc_buffer برای ذخیره‌سازی داده‌های سه کانال (۵۰ نمونه برای هر کانال) و یک پرچم volatile با نام adc_data_ready برای هماهنگی میان وقفه‌ی DMA و حلقه‌ی اصلی تعریف شده‌اند.

```
/* USER CODE BEGIN PV */
char uart_buffer[100];
uint16_t adc_buffer[150]; // 50 samples * 3 channels = 150 elements
volatile uint8_t adc_data_ready = 0; // Flag to tell the main loop the buffer
is full
/* USER CODE END PV */
```

تابع کال‌بک HAL_ADC_ConvCpltCallback در اینجا نوشته شده است. هنگامی که DMA انتقال ۱۵۰ نمونه را کامل کند، این تابع فراخوانی شده و پرچم adc_data_ready را برابر با ۱ قرار می‌دهد تا حلقه‌ی اصلی متوجه شود که داده‌های جدیدی برای پردازش آماده است.

```
/* USER CODE BEGIN 0 */
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc)
{
    // The DMA has finished filling the 150-element array
    if(hadc->Instance == ADC1)
    {
        adc_data_ready = 1;
    }
}
/* USER CODE END 0 */
```

تنها یک دستور در این بخش وجود دارد HAL_ADC_Start_DMA که ADC را در حالت اسکن پیوسته با DMA راه‌اندازی می‌کند و داده‌ها به‌طور خودکار در آرایه‌ی adc_buffer (با طول ۱۵۰) ریخته می‌شوند. این فراخوانی فقط یکبار انجام می‌شود و DMA به‌طور پیوسته در پس‌زمینه به کار خود ادامه می‌دهد.

```
/* USER CODE BEGIN 2 */
// Start the ADC in DMA mode ONE time before the while loop.
// It will run continuously in the background forever.
```

```
HAL_ADC_Start_DMA(&hadc1, (uint32_t*)adc_buffer, 150);
/* USER CODE END 2 */
```

حلقه‌ی اصلی برنامه مرتباً پرچم `adc_data_ready` را بررسی می‌کند. به محض فعال شدن آن، مجموع مقادیر مربوط به هر کانال از آرایه‌ی `adc_buffer` به‌صورت در هم آمیخته (با گام ۳: درایه‌های `i`، `i+1` و `i+2` به‌ترتیب برای کانال‌های ۱، ۲ و ۳) محاسبه می‌شود. سپس میانگین ۵۰ نمونه برای هر کانال به‌دست می‌آید. با استفاده از `sprintf` یک رشته‌ی خوانا شامل این سه میانگین ساخته شده و با `HAL_UART_Transmit` از طریق `UART1` ارسال می‌شود. در پایان پرچم صفر شده و یک تأخیر ۵۰ میلی‌ثانیه‌ای اعمال می‌گردد.

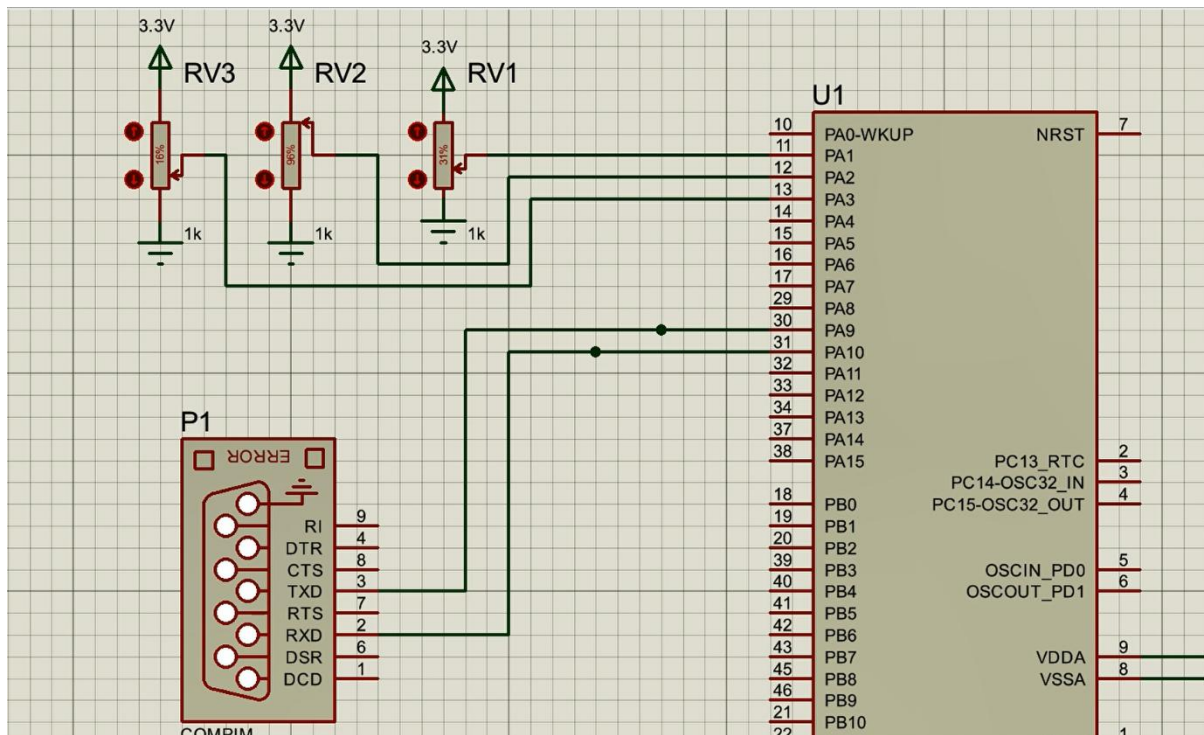
```
/* USER CODE BEGIN WHILE */
while (1)
{
    // Only process the data when the DMA tells us 50 new samples for each
    channel are ready
    if (adc_data_ready == 1)
    {
        uint32_t sum_ch1 = 0, sum_ch2 = 0, sum_ch3 = 0;
        uint16_t avg_ch1, avg_ch2, avg_ch3;

        // Step through the array. i jumps by 3 each time.
        for (int i = 0; i < 150; i += 3)
        {
            sum_ch1 += adc_buffer[i];           // Rank 1
            sum_ch2 += adc_buffer[i + 1];       // Rank 2
            sum_ch3 += adc_buffer[i + 2];       // Rank 3
        }

        avg_ch1 = sum_ch1 / 50;
        avg_ch2 = sum_ch2 / 50;
        avg_ch3 = sum_ch3 / 50;

        // Format and send the string
        sprintf(uart_buffer, "Channel 1: %04d | Channel 2: %04d | Channel 3:
        %04d\r\n", avg_ch1, avg_ch2, avg_ch3);
        HAL_UART_Transmit(&huart1, (uint8_t*)uart_buffer,
        strlen(uart_buffer), HAL_MAX_DELAY);

        // Reset the flag and wait
        adc_data_ready = 0;
        HAL_Delay(50);
    }
}
/* USER CODE END WHILE */
```



میکروکنترلر STM32F103C8 (چون کنترلر اصلی ما در داخل Proteus موجود نبود) رو در محیط شبیه سازی قرار میدهم

COMPIM را در محیط Proteus قرار می دهیم وپین ان را مطابق شکل به میکروکنترلر وصل می کنیم.

پتانسیومتر POT-HG را در محیط Proteus قرار میدهم و ان را به صورت تصویر بالا پیکر بندی می کنیم.

در اخر هم فایل Compile شده HEX. رو به کنترلر می دهیم.

4. بخش MATLAB و Virtual serial port pro

بخش Virtual Serial Port Pro مثل سوال قبل

کد متلب:

```
clc;
clear;

baud_rate = 9600;
s = serialport("COM2", baud_rate);
configureTerminator(s, "CR/LF");

disp('Reading raw data and calculating voltages... (Press Ctrl+C to stop)');
```

```

while true
    try
        % Read the incoming text line from the STM32
        str_data = readline(s);

        if strlen(str_data) > 0
            % Read the numbers out of the string using sscanf
            % This looks for the exact text format we set in the C code
            parsed_data = sscanf(str_data, "Channel 1: %d | Channel 2: %d |
Channel 3: %d");

            % If it successfully found all 3 numbers, do the math
            if length(parsed_data) == 3
                raw_ch1 = parsed_data(1);
                raw_ch2 = parsed_data(2);
                raw_ch3 = parsed_data(3);

                % Convert digital numbers to voltages
                v1 = (raw_ch1 * 3.3) / 4095.0;
                v2 = (raw_ch2 * 3.3) / 4095.0;
                v3 = (raw_ch3 * 3.3) / 4095.0;

                % Display the final output with both the raw number and the
voltage
                fprintf('CH1: %04d -> %.2f V | CH2: %04d -> %.2f V | CH3: %04d
-> %.2f V\n', ...
                    raw_ch1, v1, raw_ch2, v2, raw_ch3, v3);
            else
                % Fallback: If the string doesn't match the format exactly,
just print what arrived
                disp(str_data);
            end
        end
    end

    catch
        disp('Error reading from serial port or operation stopped. ');
        break;
    end
end
end

```

این کد متلب برای دریافت و نمایش داده‌های ارسالی از STM32 از طریق پورت سریال طراحی شده است. ابتدا پورت COM2 با نرخ ۹۶۰۰ باز شده و پایان‌دهنده‌ی خط روی CR/LF تنظیم می‌شود تا با رشته‌ی ارسالی از میکروکنترلر هماهنگ باشد. در حلقه‌ی بی‌نهایت، برنامه با `readline` یک خط کامل را دریافت کرده و سپس با `sscanf` اعداد سه کانال را از قالب مشخص‌شده (`Channel 1: %d | Channel 2: %d | Channel 3: %d`) استخراج می‌کند. اگر هر سه عدد با موفقیت خوانده شوند، مقادیر خام ADC با استفاده از فرمول

$$4095 \div (\text{مقدار} \times 3.3)$$

به ولتاژ تبدیل شده و به همراه عدد خام در پنجره‌ی فرمان نمایش داده می‌شوند. در صورتی که قالب رشته با الگوی مورد انتظار مطابقت نداشته باشد، همان خط دریافتی به‌طور خام چاپ می‌شود. بخش try-catch نیز خطاهای احتمالی در خواندن پورت سریال را مدیریت کرده و در صورت بروز مشکل، حلقه را متوقف می‌کند.

ویدیوی شبیه سازی در داخل پوشه تمرین موجود می باشد.

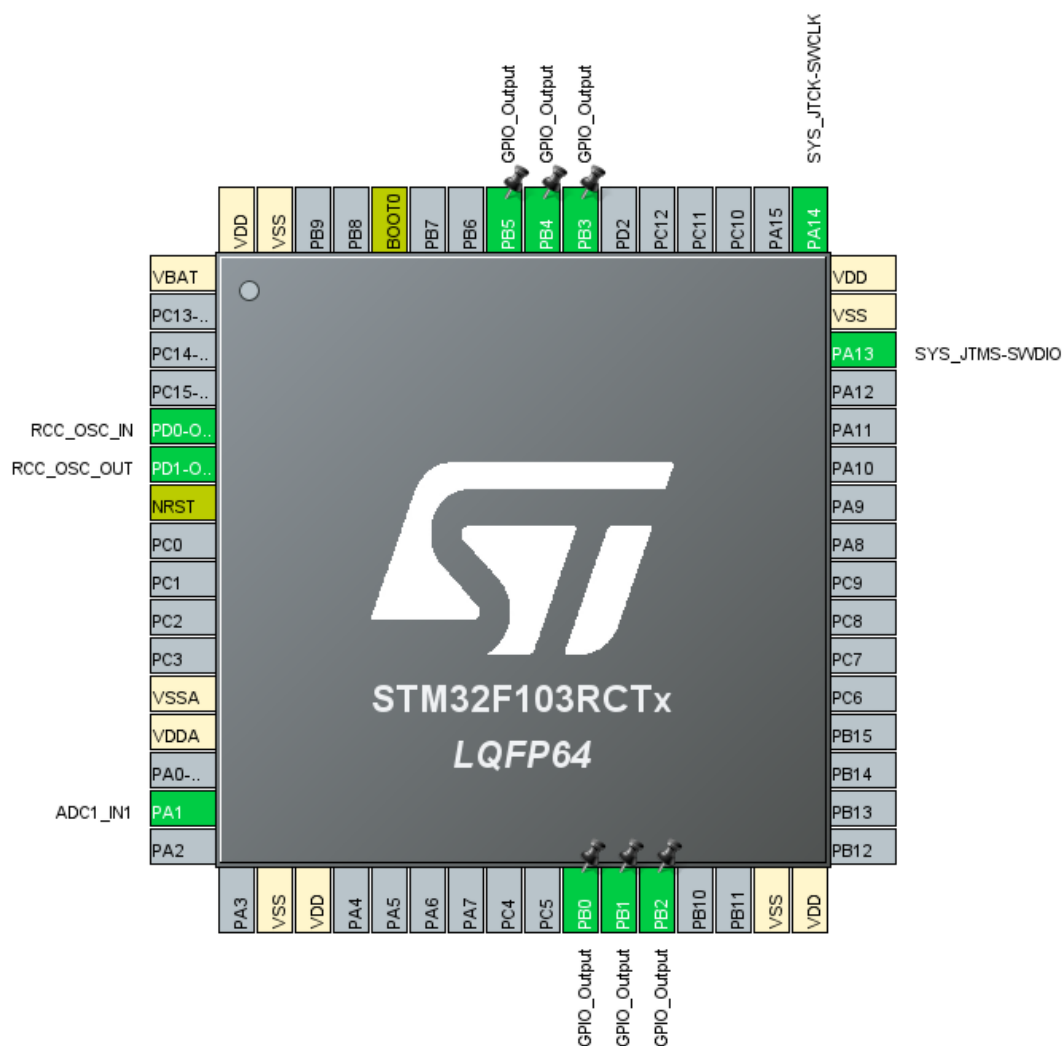
5. در یک مدار، توسط یک منبع تغذیه، ولتاژ در بازه $V[-3.3, 3.3]$ ، ایجاد نمایید. به نوعی با میکروکنترلر، این ولتاژها را در یکی از پایه های ADC خوانده و عدد مربوطه را روی LCD نمایش دهید. متناظر با عدد $V3.3$ ، عدد 4095، روی LCD نمایش داده شود و متناظر با $V3.3$ ، عدد 4095، روی LCD نمایش داده شود. با تغییر ولوم منبع تغذیه در بازه $V[-3.3, 3.3]$ ، اعداد مربوطه بصورت خطی در بازه $[-4095, 4095]$ ، تغییر نماید. دقت داشته باشید که میکروکنترلر مدنظر ما، دارای ولتاژ کاری. تا ۳٫۳ ولت است و برای حل این تمرین، لازم است ملاحظات مناسب طراحی مدار را در نظر بگیرید.

(i) بخش STM32CubeMX

میکرو کنترلر STM32F103RCT رو در نرم افزار انتخاب میکنیم.

پروگرامر رو به حالت Serial Wire و کلاک رو تنظیم میکنیم

STM32 رو مطابق تصاویر زیر پیکر بندی میکنیم.



Software Packs

Info

ADC1 Mode and Configuration

Mode

☐ IN0
☒ IN1
☐ IN2
☐ IN3
☐ IN4
☐ IN5
☐ IN6
☐ IN7
☒ IN8

Configuration

Reset Configuration

☒ Parameter Settings
 ☒ User Constants
 ☒ NVIC Settings
 ☒ DMA Settings
 ☒ GPIO Settings

Configure the below parameters :

ADC_Settings

Data Alignment

Right alignment

Scan Conversion Mode

Disabled

Continuous Conversion Mode

Disabled

Discontinuous Conversion Mode

Disabled

ADC_Regular_ConversionMode

Enable Regular Conversions

Enable

Number Of Conversion

1

External Trigger Conversion Source

Regular Conversion launched by software

Rank

1

Channel

Channel 1

Sampling Time

239.5 Cycles

ADC_Injected_ConversionMode

Enable Injected Conversions

Disable

WatchDog

Enable Analog WatchDog Mode

☐

Sampling Time

Sampling Time

Parameter Description:

Sampling Time (Cycles)

(iii) بخش Keil

فایل‌های سرآیند lcd.h و <stdio.h> به پروژه افزوده شده‌اند. اولی برای راه‌اندازی و کنترل نمایشگر LCD و دومی برای استفاده از تابع sprint در قالب‌بندی رشته‌ها به‌کار می‌روند.

```
/* USER CODE BEGIN Includes */
#include "lcd.h"
#include <stdio.h>
/* USER CODE END Includes */
```

در این بخش یک هندل از نوع Lcd_HandleTypeDef با نام lcd تعریف شده است. این هندل برای تمام تعاملات بعدی با نمایشگر LCD استفاده خواهد شد.

```
/* USER CODE BEGIN PV */
Lcd_HandleTypeDef lcd;
/* USER CODE END PV */
```

پیکربندی اولیه‌ی LCD انجام می‌شود: پایه‌های چهار خط داده (PB0 تا PB3) و دو پایه‌ی کنترلی RS (PB4) و EN (PB5) مشخص شده و یک نمونه LCD در مد ۴ بیتی ساخته می‌شود. سپس صفحه نمایشگر پاک می‌شود تا برای چاپ مقادیر آماده گردد.

```
/* USER CODE BEGIN 2 */
Lcd_PortType ports[] = { GPIOB, GPIOB, GPIOB, GPIOB };
Lcd_PinType pins[] = { GPIO_PIN_0, GPIO_PIN_1, GPIO_PIN_2, GPIO_PIN_3 };
lcd = Lcd_create(ports, pins, GPIOB, GPIO_PIN_4, GPIOB, GPIO_PIN_5,
LCD_4_BIT_MODE);

Lcd_clear(&lcd);
/* USER CODE END 2 */
```

کل منطق اندازه‌گیری و نمایش در این بخش جای گرفته است. ابتدا یک تبدیل ADC با HAL_ADC_Start آغاز شده و با HAL_ADC_PollForConversion تا پایان تبدیل صبر می‌کند. مقدار خام خوانده‌شده بلافاصله به یک متغیر با علامت ۳۲ بیتی (int32_t) ریخته می‌شود. اگر مقدار خام بیشتر از ۴۰۹۰ باشد (نویز شبیه‌سازی پروتئوس)، به ۴۰۹۵ محدود می‌شود تا مرزهای واقعی رعایت شوند. سپس مقدار دیجیتال به یک بازه‌ی دوقطبی از -۴۰۹۵ تا +۴۰۹۵ نگاشت می‌شود. $(raw_read * 2) - 4095$: ولتاژ متناظر با تقسیم این عدد بر ۴۰۹۵ و ضرب در ۳.۳ محاسبه می‌گردد. دو رشته‌ی فرمت‌شده برای نمایش ولتاژ و مقدار دیجیتال دوقطبی ساخته شده و در سطرهاى اول و دوم LCD چاپ می‌شوند. در پایان، ADC متوقف شده و ۵۰۰ میلی‌ثانیه تأخیر پیش از تکرار حلقه اعمال می‌گردد.

```
/* USER CODE BEGIN WHILE */
while (1)
{
    // 1. Start ADC, wait for it to finish
    HAL_ADC_Start(&hadc1);
```

```

if (HAL_ADC_PollForConversion(&hadc1, 10) == HAL_OK)
{
    // Cast immediately to signed 32-bit integer
    int32_t raw_read = (int32_t)HAL_ADC_GetValue(&hadc1);

    // --- PROTEUS TOLERANCE CLAMP ---
    // Simulation noise causes 100% to read ~4093. This forces it to the
true limits.
    if (raw_read > 4090)
    {
        raw_read = 4095;
    }

    // 2. Perform Mathematical Conversions
    // Map [0 to 4095] -> [-4095 to +4095]
    int32_t real_adc_value = (raw_read * 2) - 4095;

    // Map to Voltage using 3.3!
    float voltage = (real_adc_value / 4095.0) * 3.3;

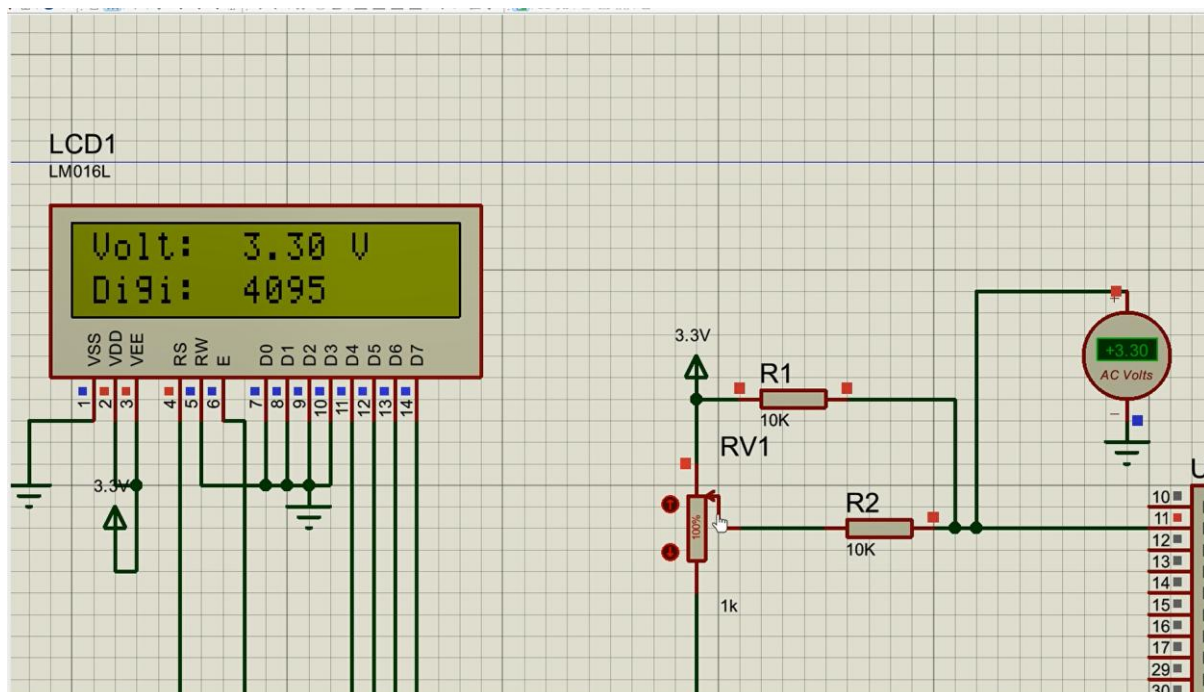
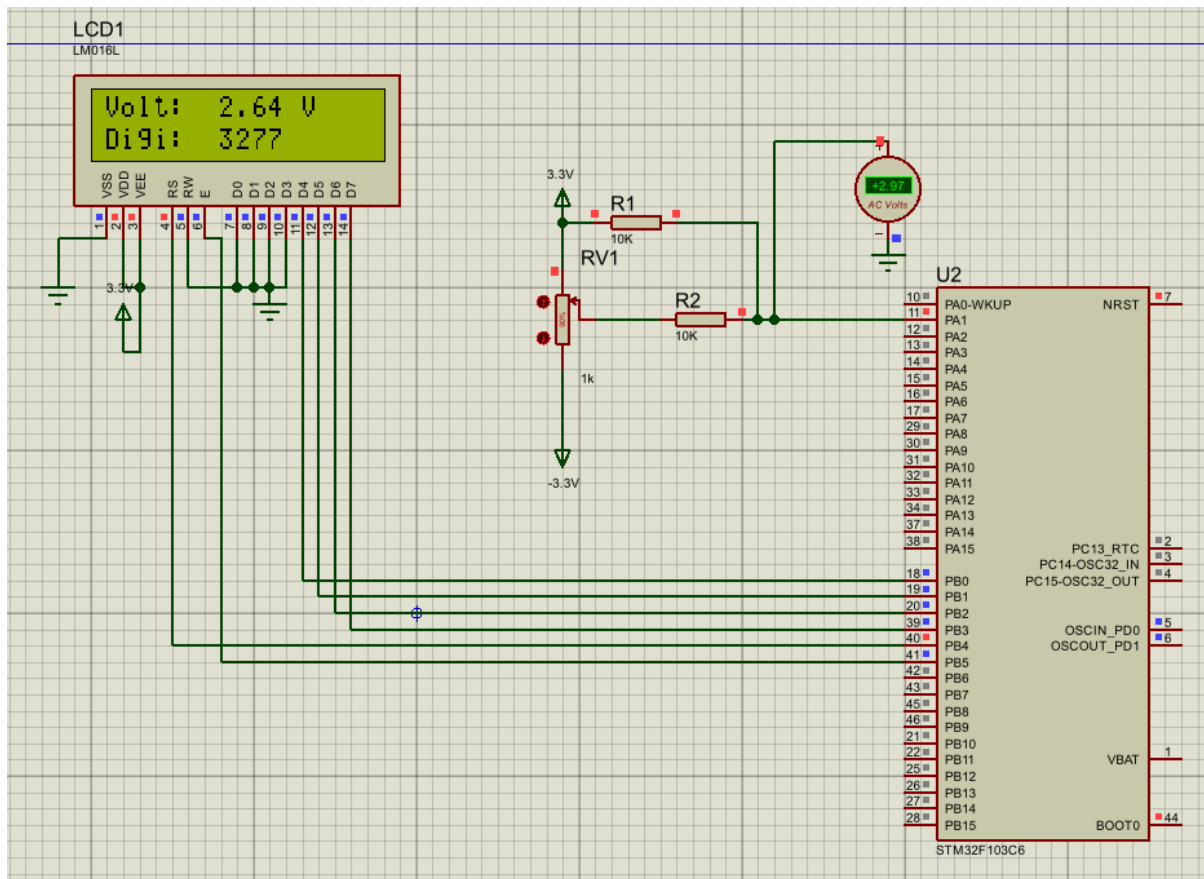
    // 3. Format the strings
    char buffer1[20];
    char buffer2[20];
    sprintf(buffer1, "Volt: %5.2f V", voltage);
    sprintf(buffer2, "Digi: %5ld", (long)real_adc_value);

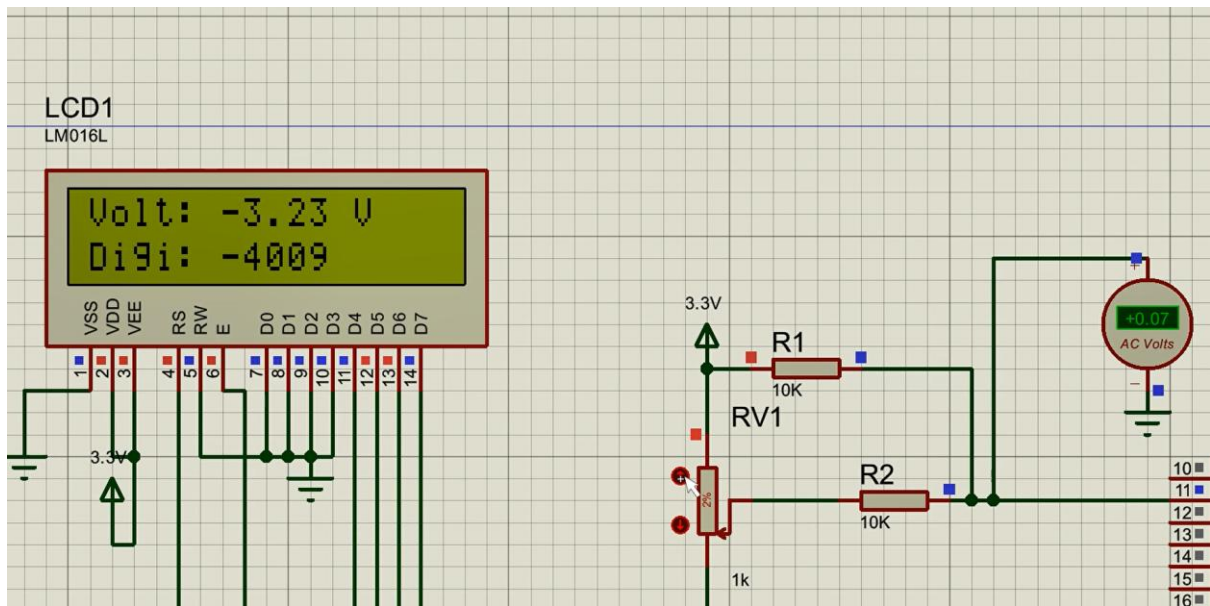
    // 4. Update the LCD Display
    Lcd_cursor(&lcd, 0, 0);
    Lcd_string(&lcd, buffer1);

    Lcd_cursor(&lcd, 1, 0);
    Lcd_string(&lcd, buffer2);
}
HAL_ADC_Stop(&hadc1);

// 5. Wait before next reading
HAL_Delay(500);
/* USER CODE END WHILE */

```





میکروکنترلر STM32F103C8 (چون کنترلر اصلی ما در داخل Proteus موجود نبود) رو در محیط شبیه سازی قرار میدهم

LCD LM016L را در محیط Proteus قرار می دهیم وپین ان را مطابق شکل به میکروکنترلر وصل می کنیم.

پتانسیومتر POT-HG را در محیط Proteus قرار میدهم و ان را به صورت تصویر بالا پیکر بندی می کنیم.

مقاومت های 10 کیلو اهم را یکی به ترمینال مثبت و دیگری را به پایه ی وسط پتانسیومتر میزیم و سر های دیگه ی مقاومت ها را به هم وصل می کنیم و از ان نود مشترک یک سیم به ADC می کشیم تا سیگنال شیفته داده شده مان را ADC بخواند , با این کار سیگنال منفی 3.3 تا مثبت 3.3 در دو سر پتانسیومتر به صفر تا مثبت 3.3 مپ می شود.

در اخر هم فایل Compile شده HEX. رو به کنترلر می دهیم.

بخش MATLAB و Virtual serial port pro

برای این سوال لازم نمی باشد.

ویدیوی شبیه سازی در داخل پوشه تمرین موجود می باشد.